

# STUDENT: Vasilije Čarapić 2022/0218

In [22]:

```
%matplotlib inline

import ipywidgets as widgets
from ipywidgets import interact, interactive, fixed, interact_manual

import skimage as ski
import numpy as np
from scipy import ndimage as ndi
from matplotlib import pyplot as plt
```

## 1) DEO

### Kreiranje maske novčića

Funkcija `coin_mask` vrši segmentaciju novčića sa slike korišćenjem HSV kolor sistema, median filtriranja i automatskog određivanja praga.

Ulazna RGB slika se konvertuje u HSV kolor sistem kako bi se informacija o boji (Hue) odvojila od osvetljenja, što omogućava stabilniju segmentaciju objekata.

Ukoliko je slika u `uint8` formatu, ona se konvertuje u `float` opseg `[0, 1]` radi pravilne obrade. Na Hue kanal se primenjuje median filter dimenzija  $13 \times 13$  sa ciljem uklanjanja šuma i zaglađivanja boje površine novčića uz očuvanje ivica.

Iz filtrirane slike se izdvaja Hue komponenta koja sadrži ključnu informaciju za razdvajanje novčića od pozadine.

Prag se određuje automatski korišćenjem Otsu metode, koja analizira histogram Hue vrednosti i pronalazi optimalnu vrednost za binarnu segmentaciju.

Binarna maska se formira poređenjem Hue vrednosti sa pragom i invertovanjem rezultata, tako da su pikseli koji pripadaju novčićima označeni belom bojom (`True`), dok je pozadina crna (`False`).

Zbog devete slike sa novčićima i nemogućnosti da se u potpunosti detektuju svi novčići (Jedan novčić desno pri grupi od 4 novčića nije mogao biti detektovan na devetoj slici) kao i zahteva da funkcija bude robusna uvedeno je da se u obzir uzme i S komponenta. Sada menjajući i H i S otvara nam se mogućnost lako detektovait tražene novčiće.

Na kraju na jednom grafiku su prikazani svi relevantni delovi zadataka. Na prvoj slici nalazi se ulazna slika, na drugoj slika koja je filtrirana medijan filtrom, na trećoj slici nalazi se histogram H vrednosti dok se na četvrtoj slici nalaze izdvojeni novčići koji su izdvojeni pomoću maske `coin_mask`.

Funkcija vraća binarnu masku novčića koja se može koristiti za dalju analizu slike kao i dodatne vrednosti koji služe za crtanje slika.

In [23]:

```
def coin_mask(img_in):
    image = ski.color.rgb2hsv(img_in)

    # Ako je uint8 -> u float
    if image.dtype == 'uint8':
        image = ski.util.img_as_float(image)

    image_med13x13 = image.copy()

    image_med13x13[:, :, 0] = ndi.median_filter(
        image[:, :, 0], size=(13, 13), mode='reflect'
    )

    # vratimo u rgb za prikaz
    med_img_rgb = ski.color.hsv2rgb(image_med13x13)

    H = image_med13x13[:, :, 0]
    S = image_med13x13[:, :, 1]

    thresh_mean = ski.filters.threshold_mean(H)
    thresh_otsu = ski.filters.threshold_otsu(H)

    thresh = thresh_otsu

    coin_mask_img = ~((H > thresh) & (S > 0.15))

    return (med_img_rgb, image_med13x13, thresh_mean, thresh_otsu, coin_mask_img)
```

In [24]:

```
def plt_coin_mask(img_in, med_img_rgb, image_med13x13, thresh_mean, thresh_otsu, coin_mask_img):
    # prikaz slike i medjukoraka
    fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(15, 5), dpi=80)
    axes[0].imshow(img_in), axes[0].set_title('Ulazna slika'), axes[0].axis('off')

    axes[1].imshow(med_img_rgb), axes[1].axis('off'), axes[1].set_title('Slika nakon medijane')

    axes[2].hist(image_med13x13.flatten(), bins=256, color='b'), axes[2].set_title('Histogram S')
    axes[2].axvline(x=thresh_mean, color='r', linestyle='--')
    axes[2].axvline(x=thresh_otsu, color='g', linestyle='dashdot')
    axes[2].set_facecolor('white')
    axes[2].legend(['mean '+'{:.2f}'.format(thresh_mean), 'Otsu '+'{:.2f}'.format(thresh_otsu)])

    axes[3].imshow(coin_mask_img, cmap='gray'), axes[3].axis('off'), axes[3].set_title('Maska novčića')

    plt.tight_layout()
    plt.show()
```

Radi validnog izbora **S** komponente tako da na najbolji način obuhvatimo sve novčiće na način na koji neće imati rupice napisana je slična funkcija samo je njena svrha da se koristi

`widgets.interact` kako bi se na najbolji način odredio parametar `S`.

Nakon testiranja više varijanti dolazi se do zaključka da je za  **$S = 0.15$**  najbolja vrednost, pa je slika izdvojena sa tom vrednošću koja je uzeta kao parametar u pravljenju maske u gore pomenutoj funkciji.

```
In [ ]: # def coin_mask_test(img_in, S_val):
#         image = ski.color.rgb2hsv(img_in)

#         # Ako je uint8 → u float
#         if image.dtype == 'uint8':
#             image = ski.util.img_as_float(image)

#         image_med13x13 = image.copy()

#         image_med13x13[:, :, 0] = ndi.median_filter(
#             image[:, :, 0], size=(13, 13), mode='reflect'
#         )

#         H = image_med13x13[:, :, 0]
#         S = image_med13x13[:, :, 1]
#         thresh_mean = ski.filters.threshold_mean(H)
#         thresh_otsu = ski.filters.threshold_otsu(H)

#         thresh = thresh_otsu

#         coin_mask = ~((H > thresh) & (S > S_val))

#         # prikaz slike i medjukoraka
#         fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(15, 5), dpi=80)
#         axes[0].imshow(image), axes[0].set_title('Ulazna slika'), axes[0].axis('off')

#         axes[1].imshow(image_med13x13), axes[1].axis('off'), axes[1].set_title('Slika r

#         axes[2].hist(image_med13x13.flatten(), bins=256, color='b'), axes[2].set_title(
#         axes[2].axvline(x=thresh_mean, color='r', linestyle='-')
#         axes[2].axvline(x=thresh_otsu, color='g', linestyle='dashdot')
#         axes[2].set_facecolor('white')
#         axes[2].legend(['mean '+'{:0.2f}'.format(thresh_mean), 'Otsu '+'{:0.2f}'.format(1

#         axes[3].imshow(coin_mask, cmap='gray'), axes[3].axis('off'), axes[3].set_title(

#         plt.tight_layout()
#         plt.show()
```

```
In [ ]: # for i in range(1, 10):
#         img = ski.io.imread('../sekvence/coins{}.jpg'.format(i))
#         widgets.interact(coin_mask_test, img_in = fixed(img), S_val = (0,1,0.01))
```

Ova ćelija prikazuje rezultate i medjurezltate funkcije `coin_mask` za svih 9 slika redom od prve do poslednje slike.

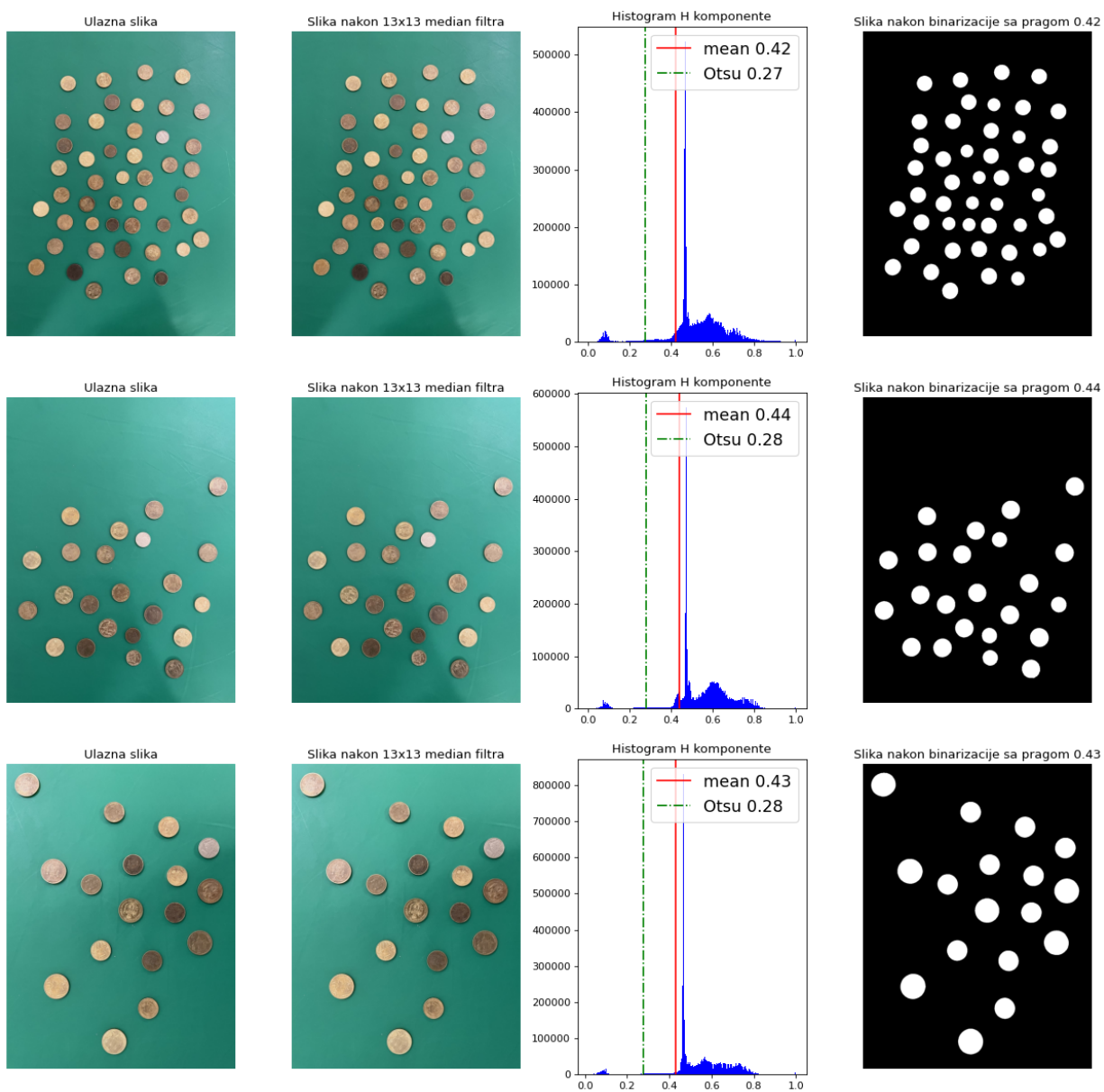
Zbog činjenice da se 9 slika odjednom u listi prikazuje kao i da se funkcija `coin_mask` ne izvršava trenutno potrebno je određeno vreme da bi se ova ćelija izvršila.

In [25]:

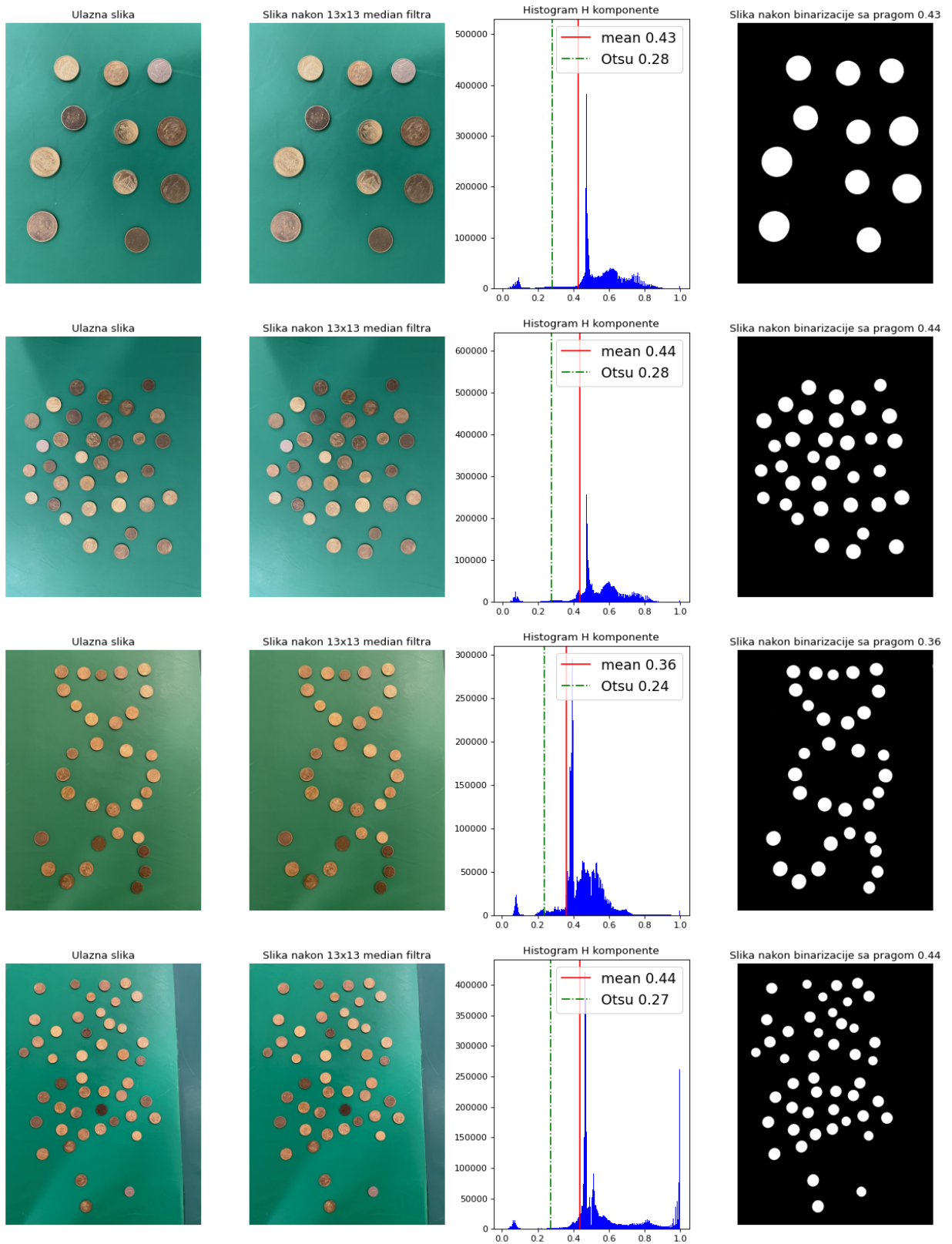
```

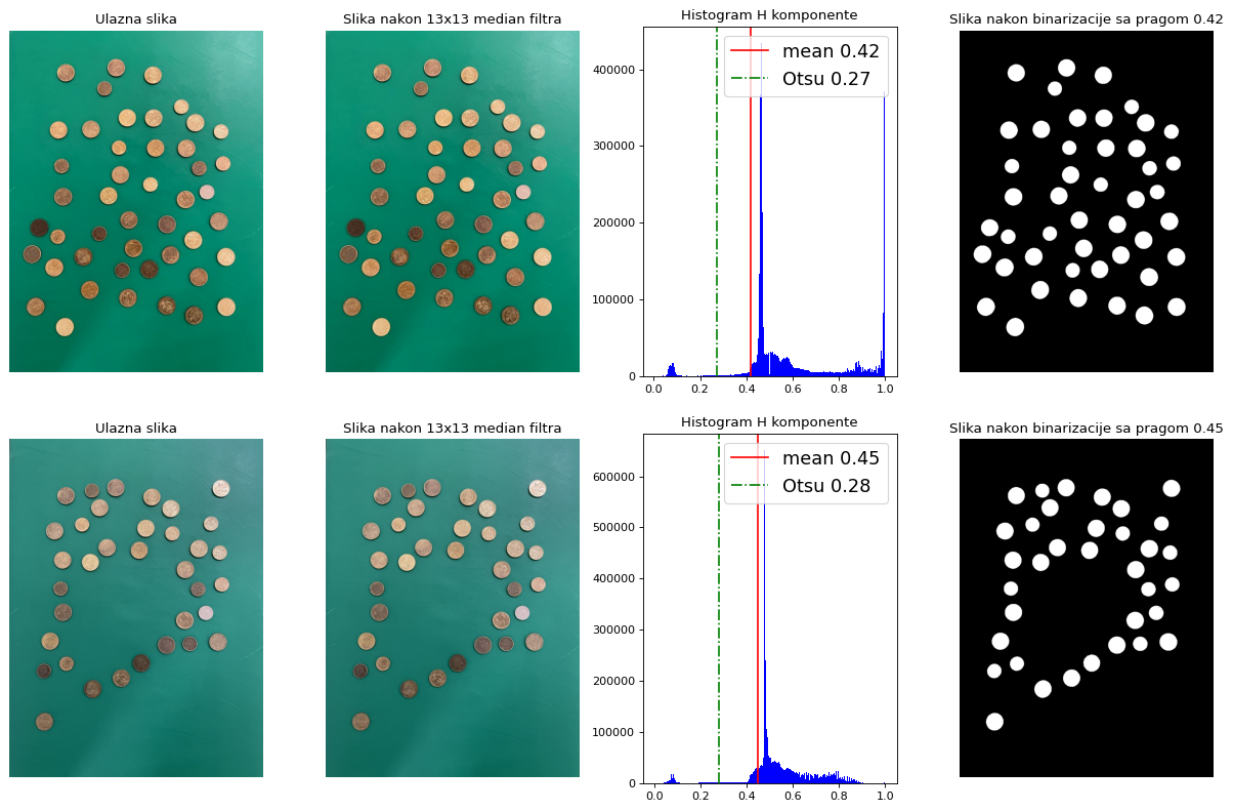
images, masks = [], []
for i in range(1, 10):
    img = ski.io.imread('../sekvence/coins{}.jpg'.format(i))
    images.append(img)
    med_img_rgb, image_med13x13, thresh_mean, thresh_otsu, coin_mask_img = coin_mask(
    masks.append(coin_mask_img)
    plt_coin_mask(img, med_img_rgb, image_med13x13, thresh_mean, thresh_otsu, coin_ma

```









b)

Funkcija `bw_label` označava sve **povezane regione (connected components)** na binarnoj slici tako što svakom povezanom delu dodeljuje jedinstvenu labelu. Funkcija radi na sledeći način:

### 1. Kreiranje inicijalne slike labela ( `bw_img` )

Napravimo matricu iste dimenzije kao ulazna binarna slika `img_bin`. Svaki piksel koji je 1 (beli piksel) dobija jedinstvenu labelu, dok crni pikseli (imaju vrednost nula) ostaju 0. Na ovaj način svaki piksel počinje sa svojom inicijalnom, privremenom vrednošću.

### 2. Priprema koordinatne mreže ( `np.meshgrid` )

Generišemo dve matrice `X` i `Y` koje sadrže X i Y koordinate svakog piksela slike.

### 3. Definisanje suseda (8-komšija)

Napravimo listu `pixel_nghs` koja sadrži pomake za svih 8 suseda jednog piksela (gore, dole, levo, desno i dijagonale).

### 4. Iterativna propagacija labela

- Pretpostavimo da se više neće menjati ( `stop_loop = True` ).
- Za svakog komšiju trenutnog piksela:
  - A. Izračunamo koordinate suseda i ograničimo ih na validne granice slike ( `np.clip` ), kako ne bismo izašli van opsega.
  - B. Napravimo masku piksela gde uzimmo pozicije gde trenutni i susedni piksel nisu nula.
  - C. Za te piksele računamo novu vrednost kao minimum između trenutne i susedove

labele.

D. Ako je došlo do promene vrednosti piksela, označimo da je potrebno nastaviti kretati se kroz petlju ( `stop_loop = False` ).

## 5. Zaustavljanje petlje

- Petlja se ponavlja sve dok **nijedan piksel ne promeni svoju labelu**, to jest kada svaki piksel u 8-susedstvu ima najmanju moguću labelu.

Funkcija vraća matricu `bw_img` u kojoj svi pikseli iste povezanosti imaju istu labelu.

In [26]:

```
def bw_label(img_bin):
    H, W = img_bin.shape
    X, Y = np.meshgrid(np.arange(H), np.arange(W), indexing='ij')

    # X mi ide nadole Y mi ide nadesno
    # inicijalne labele
    bw_img = np.zeros_like(img_bin, dtype=int)
    num_of_white = np.sum(img_bin == 1)

    # tamo gde je img_bin == 1 toliko razlicitih labela dobijaju pikseli
    bw_img[img_bin == 1] = np.arange(1, num_of_white + 1)

    # definicija 8-komsija
    pixel_nghs = [(-1, -1), (-1, 0), (-1, 1),
                  (0, -1),          (0, 1),
                  (1, -1),  (1, 0),  (1, 1)]

    # bw_img slika koja mi ima sve razlicite labele gde su kecevi
    while True:
        stop_loop = True # pretpostavimo da nema promena

        # prolazak kroz sve komsije
        for dx, dy in pixel_nghs:
            # dodajemo dx/dy i koristimo clip da ne izlazimo iz opsega
            Xn = np.clip(X + dx, 0, H - 1)
            Yn = np.clip(Y + dy, 0, W - 1)

            # maska = (trenutni piksel nije 0) AND (komsija nije 0)
            mask = (bw_img[X, Y] != 0) & (bw_img[Xn, Yn] != 0)

            # uzmi manju vrednost od obe labele koje posmatram i stavi mi tu labelu
            new_values = np.minimum(bw_img[X[mask], Y[mask]], bw_img[Xn[mask], Yn[mask]])

            # ako se neka vrednost promenila, update-uj i oznaci da još nismo završili
            if np.any(new_values != bw_img[X[mask], Y[mask]]):
                bw_img[X[mask], Y[mask]] = new_values
                stop_loop = False # promena detektovana

        # ako nijedan piksel nije promenjen -> izlazimo iz while
        if stop_loop:
            break

    return bw_img
```

Ovde je ostavljena implementacija funkcije `bw_label` koja koristi ugrađenu funkciju

`skimage.measure.label` koja kada se pokrene umesto funkcije istog imena koja je ručno napisana će dati iste rezultate ali značajno brže i njena uloga je za proveru da rezultati rade kako treba.

Ukoliko se žele proveriti rezultati sa ovom funkcijom potrebno je otkomentarirati ćeliju dole i pokrenuti je ali tako da je ta funkcija ugrađena sa istim imenom poslednja pokrenuta jer će tada biti funkcije otkomentarisane obe sa istim imenom pa se računa ona funkcija koja je poslednja pokrenuta.

```
In [ ]: # brza funkcija ugrađena u cu
# def bw_label(img_bin):
#     bw_img = ski.measure.Label(img_bin, connectivity=2)
#     return bw_img
```

Nakon što smo laberirali sve segmente koji imaju vrednost 1, pomoću funkcija `color_labels` i `plt_labels` laberiramo sve labele različitom bojom tako da je svaki novčić obojen različitom bojom.

```
In [27]: def color_labels(bw_img, orig_img):
regions = ski.measure.regionprops(bw_img)

coin_areas = {}

#Prolazak kroz sve objekte i ispis parametara Area
for i, region in zip(range(len(regions)), regions):
    coin_areas["coin{}_area".format(i)] = region["Area"]

return coin_areas, regions
```

```
In [28]: def plt_labels(orig_img, bw_img, regions):
#crtanje slika
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15, 7), dpi=80)
axes[0].imshow(orig_img), axes[0].set_title('Ulazna slika'), axes[0].axis('off')

image_label_overlay = ski.color.label2rgb(bw_img, image=orig_img, bg_label=0)
axes[1].imshow(image_label_overlay, cmap='jet'), axes[1].set_title('labelirana slika')
for i, region in zip(range(len(regions)), regions):
    axes[1].annotate(text=str(region['Area']), xy=(region['Centroid'][1], region['Centroid'][0]),
                    horizontalalignment='center', verticalalignment='top', fontsize=12)

plt.tight_layout()

#pretvaranje u sliku za cuvanje za treci deo
fig.canvas.draw()
output_img = np.frombuffer(fig.canvas.tostring_rgb(), dtype=np.uint8)
output_img = output_img.reshape(fig.canvas.get_width_height()[::-1] + (3,))

plt.show()

return output_img
```

Sada se testira funkcionalnost funkcija iz tačke b) na sledeći način:

1. Učitamo binarizovane slike i originalne slike i smestimo ih u promenljive `coin_mask` i `img` sa odgovarajućim brojem
2. primenimo funkciju `bw_label` da bi dodelili labelu određenim slikama
3. nakon dodele labela primenimo funkciju `color_labels` pomoću koje obavljam obajanje labela u određene boje i prikazivanje rezultata
4. Na kraju upoređujemo rezultate broja novčića na slici od slike originalne i slike koja je estimirana sa labelama pomoću segmentacije

## Važna napomena

Na nekim slikama (primarno na **slici 6**) javljaju se artefakti (u desnom gornjem uglu za **sliku 6**) koje nije bilo moguće u potpunosti ukloniti.

Zbog toga je u tački **1.c**) neophodno primeniti dodatne korake filtriranja kako bi se obezbedilo tačno prebrojavanje novčića i sprečio uticaj artefakata na segmentaciju.

In [29]:

```
index = 0
coin_mask1, img1 = masks[index], images[index]
coin_mask2, img2 = masks[index + 1], images[index + 1]
coin_mask3, img3 = masks[index + 2], images[index + 2]
coin_mask4, img4 = masks[index + 3], images[index + 3]
coin_mask5, img5 = masks[index + 4], images[index + 4]
coin_mask6, img6 = masks[index + 5], images[index + 5]
coin_mask7, img7 = masks[index + 6], images[index + 6]
coin_mask8, img8 = masks[index + 7], images[index + 7]
coin_mask9, img9 = masks[index + 8], images[index + 8]

bw_img1 = bw_label(coin_mask1)
bw_img2 = bw_label(coin_mask2)
bw_img3 = bw_label(coin_mask3)
bw_img4 = bw_label(coin_mask4)
bw_img5 = bw_label(coin_mask5)
bw_img6 = bw_label(coin_mask6)
bw_img7 = bw_label(coin_mask7)
bw_img8 = bw_label(coin_mask8)
bw_img9 = bw_label(coin_mask9)
```

In [30]:

```
coin_areas1, regions1 = color_labels(bw_img1, img1)
coin_areas2, regions2 = color_labels(bw_img2, img2)
coin_areas3, regions3 = color_labels(bw_img3, img3)
coin_areas4, regions4 = color_labels(bw_img4, img4)
coin_areas5, regions5 = color_labels(bw_img5, img5)
coin_areas6, regions6 = color_labels(bw_img6, img6)
coin_areas7, regions7 = color_labels(bw_img7, img7)
coin_areas8, regions8 = color_labels(bw_img8, img8)
coin_areas9, regions9 = color_labels(bw_img9, img9)

oi1 = plt_labels(img1, bw_img1, regions1)
oi2 = plt_labels(img2, bw_img2, regions2)
oi3 = plt_labels(img3, bw_img3, regions3)
oi4 = plt_labels(img4, bw_img4, regions4)
oi5 = plt_labels(img5, bw_img5, regions5)
oi6 = plt_labels(img6, bw_img6, regions6)
oi7 = plt_labels(img7, bw_img7, regions7)
oi8 = plt_labels(img8, bw_img8, regions8)
oi9 = plt_labels(img9, bw_img9, regions9)

coin_areas = [
    coin_areas1,
    coin_areas2,
    coin_areas3,
    coin_areas4,
    coin_areas5,
    coin_areas6,
    coin_areas7,
    coin_areas8,
    coin_areas9
]

img_table_values = []
for iterable in coin_areas:
    img_table_values.append(len(iterable.keys()))

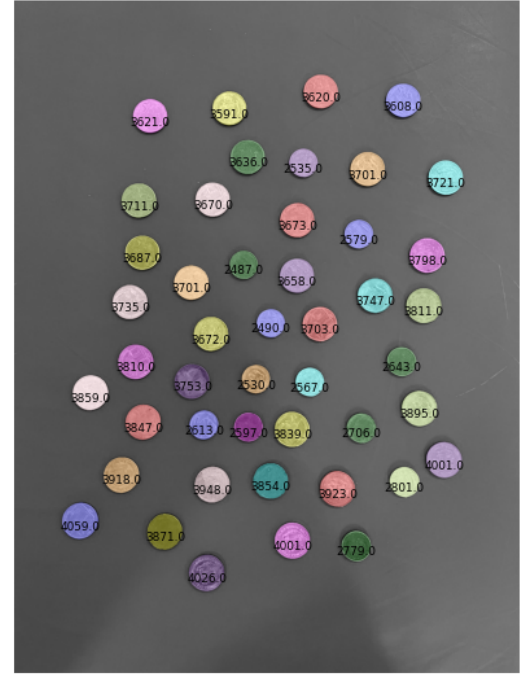
real_table_values = [46, 23, 17, 11, 34, 32, 46, 46, 35]

curr_img = 1
for tb_val1, tb_val2 in zip(img_table_values, real_table_values):
    print("Image {}:".format(curr_img))
    print("Procenjen broj novčića:{}, Pravi broj novčića:{}".format(tb_val1, tb_val2))
    curr_img += 1
```

Ulazna slika



labelirana slika



Ulazna slika



labelirana slika





Ulazna slika



labelirana slika



Ulazna slika



labelirana slika





Ulazna slika



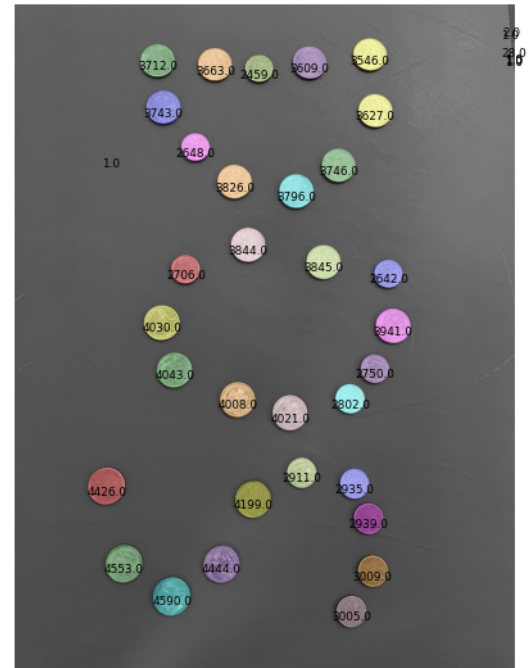
labelirana slika



Ulazna slika



labelirana slika



Ulazna slika



labelirana slika



Ulazna slika



labelirana slika





Image 1:

Procenjen broj novčića:46, Pravi broj novčića:46

Image 2:

Procenjen broj novčića:23, Pravi broj novčića:23

Image 3:

Procenjen broj novčića:17, Pravi broj novčića:17

Image 4:

Procenjen broj novčića:12, Pravi broj novčića:11

Image 5:

Procenjen broj novčića:34, Pravi broj novčića:34

Image 6:

Procenjen broj novčića:39, Pravi broj novčića:32

Image 7:

Procenjen broj novčića:46, Pravi broj novčića:46

Image 8:

Procenjen broj novčića:46, Pravi broj novčića:46

Image 9:

Procenjen broj novčića:36, Pravi broj novčića:35

Na osnovu dobijenih rezultata može se zaključiti da povećan broj detektovanih novčića na pojedinim slikama nije posledica šuma, već neidealne binarizacije slike. Primenjeni globalni prag (Otsu metoda) u određenim slučajevima ostavlja male povezane regione koji ne pripadaju stvarnim novčićima, ali se u binarnoj slici pojavljuju kao pikseli vrednosti jedan.

Tokom primene funkcije `bw_label`, ovi mali regioni bivaju označeni kao zasebne labele, što dovodi do precenjivanja broja novčića u fazi segmentacije. Iako je histogram u većini slučajeva dovoljan za izdvajanje novčića, na pojedinim slikama (npr. slika 6) javlja se odstupanje koje zahteva dodatnu obradu, kako bi se uklonili neželjeni regioni i obezbedila tačnost prebrojavanja.

```
In [ ]: # print("Image 1:\n", coin_areas1)
# print("Image 2:\n", coin_areas2)
# print("Image 3:\n", coin_areas3)
# print("Image 4:\n", coin_areas4)
# print("Image 5:\n", coin_areas5)
# print("Image 6:\n", coin_areas6)
# print("Image 7:\n", coin_areas7)
# print("Image 8:\n", coin_areas8)
# print("Image 9:\n", coin_areas9)
```

v)

Funkcija `coin_classification` klasifikuje novčiće na one od 1 dinara i na one od 5 dinara.

**Crvenom** bojom su označeni novčići od **5 dinara**, dok su **plavom** bojom označeni novčići od **1 dinara**.

Postupak realizovanja funkcije je sledeći:

- Binarizujemo ulaznu sliku.
- Labeliramo ulaznu sliku.
- Izračunamo površine na slici i u rečnik **coin\_areas** dobijamo površine svih objekata.
- Pretpostavimo da ne postoje novčići sa površinom manjom od određenog broja (npr. 400) i sve površine manje od toga uklonimo iz rečnika.
- Od preostalih površina izračunamo srednju vrednost pomoću formule:

$$\text{coinThresh} = \frac{\min(\text{areaValues}) + \max(\text{areaValues})}{2}$$

- Svaki piksel koji je manji od praga označavamo kao **1 dinar**, a svaki piksel koji je veći od praga označavamo kao **5 dinara**.
- Ako je 5 dinara, bojimo u **crveno**, dok ako je 1 dinar, bojimo u **plavo**.
- Na kraju prikazujemo rezultate na slici i ispisujemo kojoj klasi pripada preko svakog novčića.

In [31]:

```
def coin_classification(input_img):
    # binarizovana slika novčica
    _, _, _, bin_img = coin_mask(input_img)

    # labelirana slika
    bw_img = bw_label(bin_img)

    # dobija površine po labeli
    coin_areas, label_regions = color_labels(bw_img, input_img)

    # ukloni artefakte
    min_area_threshold = 400
    keys_to_delete = [k for k in coin_areas if coin_areas[k] < min_area_threshold]
    for k in keys_to_delete:
        del coin_areas[k]

    # prag po površini
    area_values = list(coin_areas.values())
    coin_thresh = (min(area_values) + max(area_values)) / 2

    # klasifikacija po površini
    classes = []
    for area in coin_areas.values():
        if area > coin_thresh:
            classes.append(5)
        else:
            classes.append(1)

    # brojanje
    num_1 = classes.count(1)
    num_5 = classes.count(5)

    # regioni bez artefakata
    regions = [r for r in ski.measure.regionprops(bw_img) if r.area > min_area_thresh]

    # bojenje po klasi
    colored = np.zeros((*bw_img.shape, 3), dtype=float)
    for region, c in zip(regions, classes):
        if c == 5:
            colored[bw_img == region.label] = [1, 0, 0] # crveno za 5 din
        else:
            colored[bw_img == region.label] = [0, 0, 1] # plavo za 1 din

    # overlay sa originalom (da se vidi i original)
    image_overlay = (0.4 * input_img/255.0) + (0.6 * colored)
    image_overlay = np.clip(image_overlay, 0, 1)

    return [num_1, num_5], bw_img, label_regions, image_overlay, classes, regions
```

In [32]:

```
def plot_final_coins(image_overlay, regions, classes):
    # crtanje slike
    fig, axes = plt.subplots(figsize=(12, 8), dpi=80)
    axes.imshow(image_overlay)
    axes.axis('off')

    # annotate
    for region, c in zip(regions, classes):
        axes.annotate(
            text=str(c),
            xy=(region.centroid[1], region.centroid[0]),
            xytext=(region.centroid[1], region.centroid[0]),
            horizontalalignment='center',
            verticalalignment='top',
            fontsize=8
        )

    plt.tight_layout()

    # pretvaranje figure u sliku da bi je sacuvali posle
    fig.canvas.draw()
    output_img = np.frombuffer(fig.canvas.tostring_rgb(), dtype=np.uint8)
    output_img = output_img.reshape(fig.canvas.get_width_height()[::-1] + (3,))

    plt.show()

    return output_img
```

In [33]:

```
class_img1, bw_img1, label_regions1, image_overlay1, classes1, regions1 = coin_class:
class_img2, bw_img2, label_regions2, image_overlay2, classes2, regions2 = coin_class:
class_img3, bw_img3, label_regions3, image_overlay3, classes3, regions3 = coin_class:
class_img4, bw_img4, label_regions4, image_overlay4, classes4, regions4 = coin_class:
class_img5, bw_img5, label_regions5, image_overlay5, classes5, regions5 = coin_class:
class_img6, bw_img6, label_regions6, image_overlay6, classes6, regions6 = coin_class:
class_img7, bw_img7, label_regions7, image_overlay7, classes7, regions7 = coin_class:
class_img8, bw_img8, label_regions8, image_overlay8, classes8, regions8 = coin_class:
class_img9, bw_img9, label_regions9, image_overlay9, classes9, regions9 = coin_class:

class1, class5 = (12, 4, 10, 7, 12, 11, 12, 12, 12), (34, 19, 7, 4, 22, 21, 34, 34, 1

# stampanje tabela koliko jedinica i petica ima u slici
class_imgs = [
    class_img1, class_img2, class_img3,
    class_img4, class_img5, class_img6,
    class_img7, class_img8, class_img9
]

for i, cls in enumerate(class_imgs):
    print(
        f"Img{i+1}: 1: {cls[0]} 5: {cls[1]} , "
        f"Orig Img{i+1}: 1: {class1[i]} 5: {class5[i]}"
    )
```

Img1: 1: 12 5: 34 , Orig Img1: 1: 12 5: 34

Img2: 1: 4 5: 19 , Orig Img2: 1: 4 5: 19

```
Img3: 1: 10 5: 7 , Orig Img3: 1: 10 5: 7
Img4: 1: 7 5: 4 , Orig Img4: 1: 7 5: 4
Img5: 1: 12 5: 22 , Orig Img5: 1: 12 5: 22
Img6: 1: 11 5: 21 , Orig Img6: 1: 11 5: 21
Img7: 1: 12 5: 34 , Orig Img7: 1: 12 5: 34
Img8: 1: 12 5: 34 , Orig Img8: 1: 12 5: 34
Img9: 1: 12 5: 23 , Orig Img9: 1: 12 5: 23
```

---

### Analiza i objašnjenje:

- Za većinu slika detekcija je identična originalu, što potvrđuje **pouzdanost funkcije `coin_classification`** i izbacivanje artefakta prethodno navedenom metodom.
- Broj novčića klase 1 i 5 je pravilno klasifikovan jer:
  - Binarizacija slike i pragovi (Otsu) pravilno odvajaju novčiće od pozadine.
  - Funkcija `bw_label` dodeljuje labele svim povezanim regionima.
  - Filtriranje regiona po površini eliminiše male šumove, što sprečava precenjivanje broja novčića.
- Ovi rezultati pokazuju da su kombinacija **binarizacije, labeliranja i klasifikacije po površini** dobro podešeni i da algoritam radi stabilno za sve slike iz skupa.

### Zaključak:

Algoritam pouzdano detektuje i klasifikuje novčiće, broj novčića i klasifikacija se poklapaju sa originalnim vrednostima, što potvrđuje njegovu tačnost i primenljivost.

---

## Medjurezultati za dve funkcije

### Prikaz i čuvanje rezultata

#### Objasnenje:

- Kod prikazuje kompletan postupak obrade za **prvu** i **šestu** sliku.
- Najpre se funkcijom `coin_mask` dobijaju binarne maske novčića i pragovi za svaku sliku.
- Zatim se funkcijom `plt_coin_mask` prikazuju međurezultati binarizacije.
- Nakon toga se vrši labeliranje objekata pomoću `plt_labels`.
- Funkcija `plot_final_coins` prikazuje konačnu klasifikaciju novčića na 1 i 5 dinara.

Na kraju se rezultati labeliranja i finalne slike sa klasifikovanim novčićima čuvaju kao `.jpg` fajlovi.



In [34]:

```

# prva slika
med_img_rgb1, image_med13x13_1, thresh_mean1, thresh_otsu1, coin_mask_img1 = coin_mask(
plt_coin_mask(img1, med_img_rgb1, image_med13x13_1, thresh_mean1, thresh_otsu1, coin_

out_label1 = plt_labels(img1, bw_img1, label_regions1)

out_coin1 = plot_final_coins(image_overlay1, regions1, classes1)

# sesta slika
med_img_rgb6, image_med13x13_6, thresh_mean6, thresh_otsu6, coin_mask_img6 = coin_mask(
plt_coin_mask(img6, med_img_rgb6, image_med13x13_6, thresh_mean6, thresh_otsu6, coin_

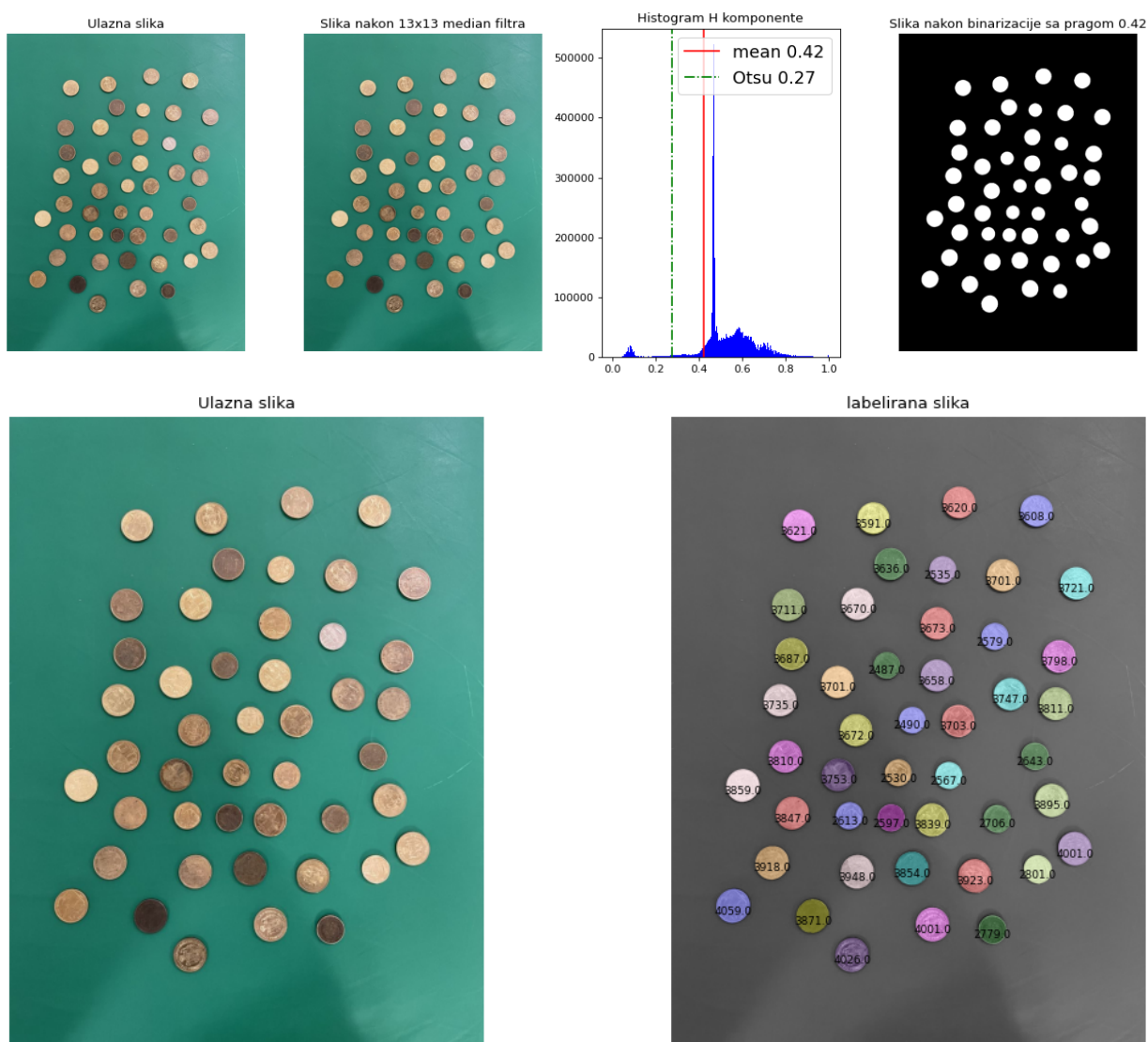
out_label6 = plt_labels(img6, bw_img6, regions6)

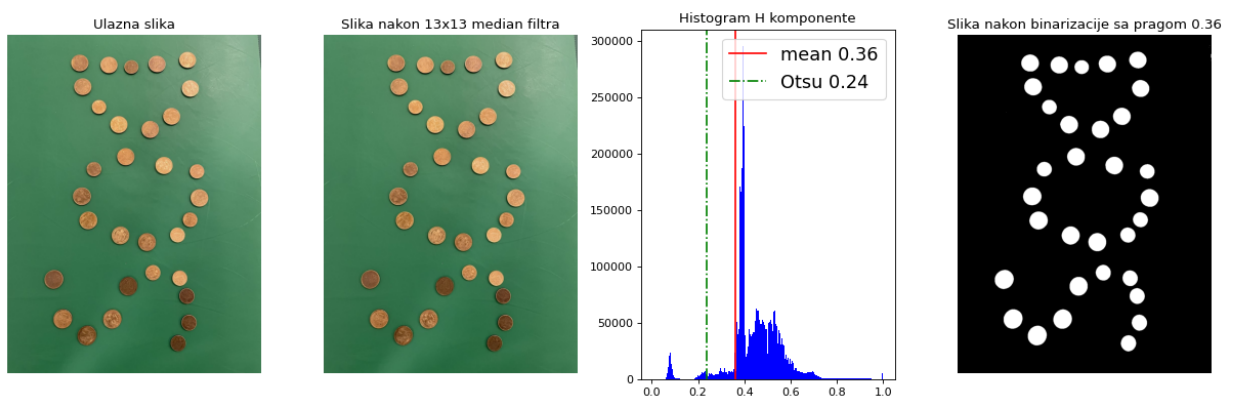
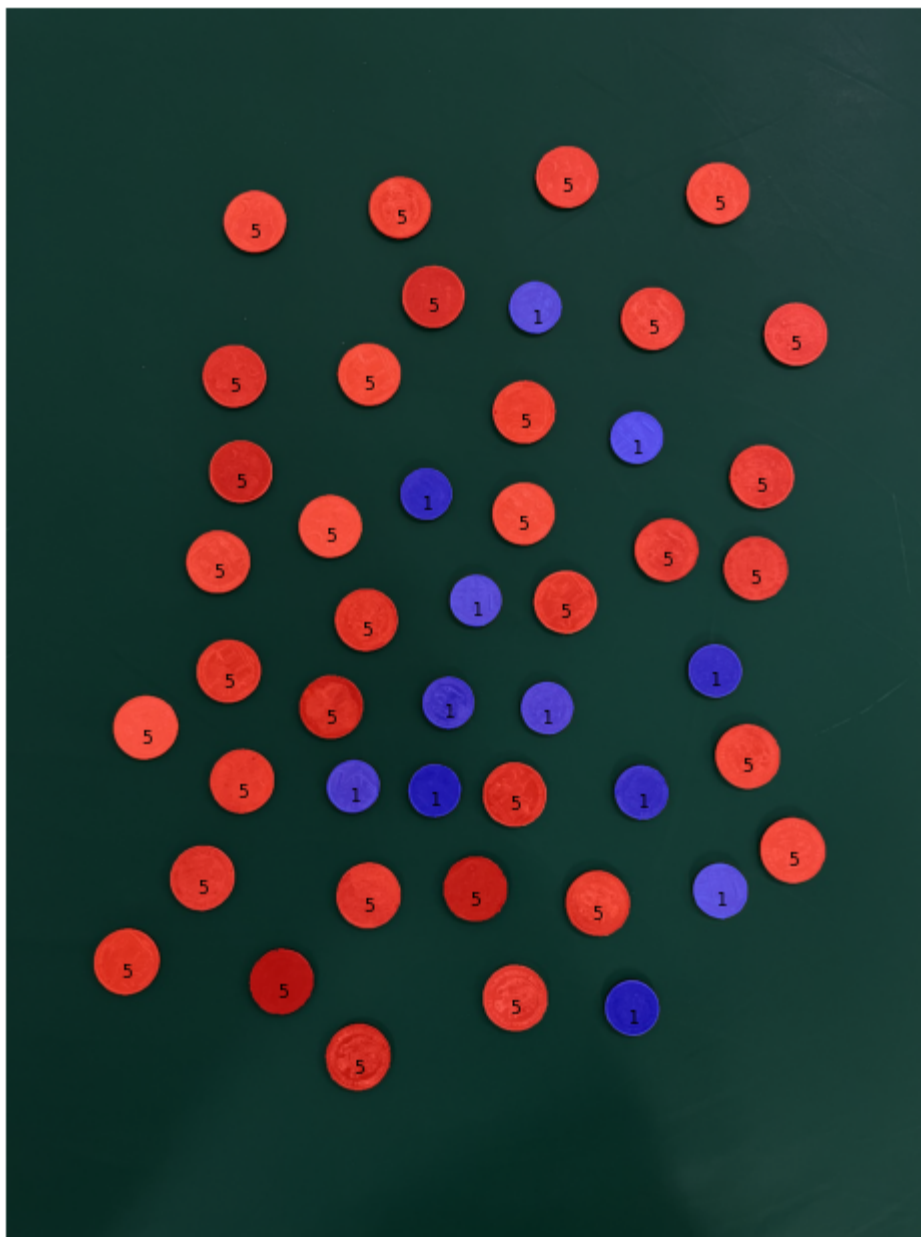
out_coin6 = plot_final_coins(image_overlay6, regions6, classes6)

ski.io.imsave("imgLabel1.jpg", out_label1)
ski.io.imsave("imgLabel6.jpg", out_label6)

ski.io.imsave("imgCoins1.jpg", out_coin1)
ski.io.imsave("imgCoins6.jpg", out_coin6)

```

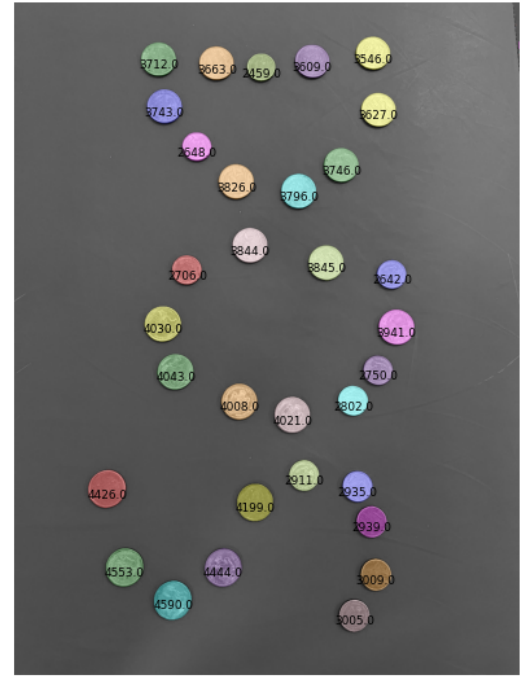


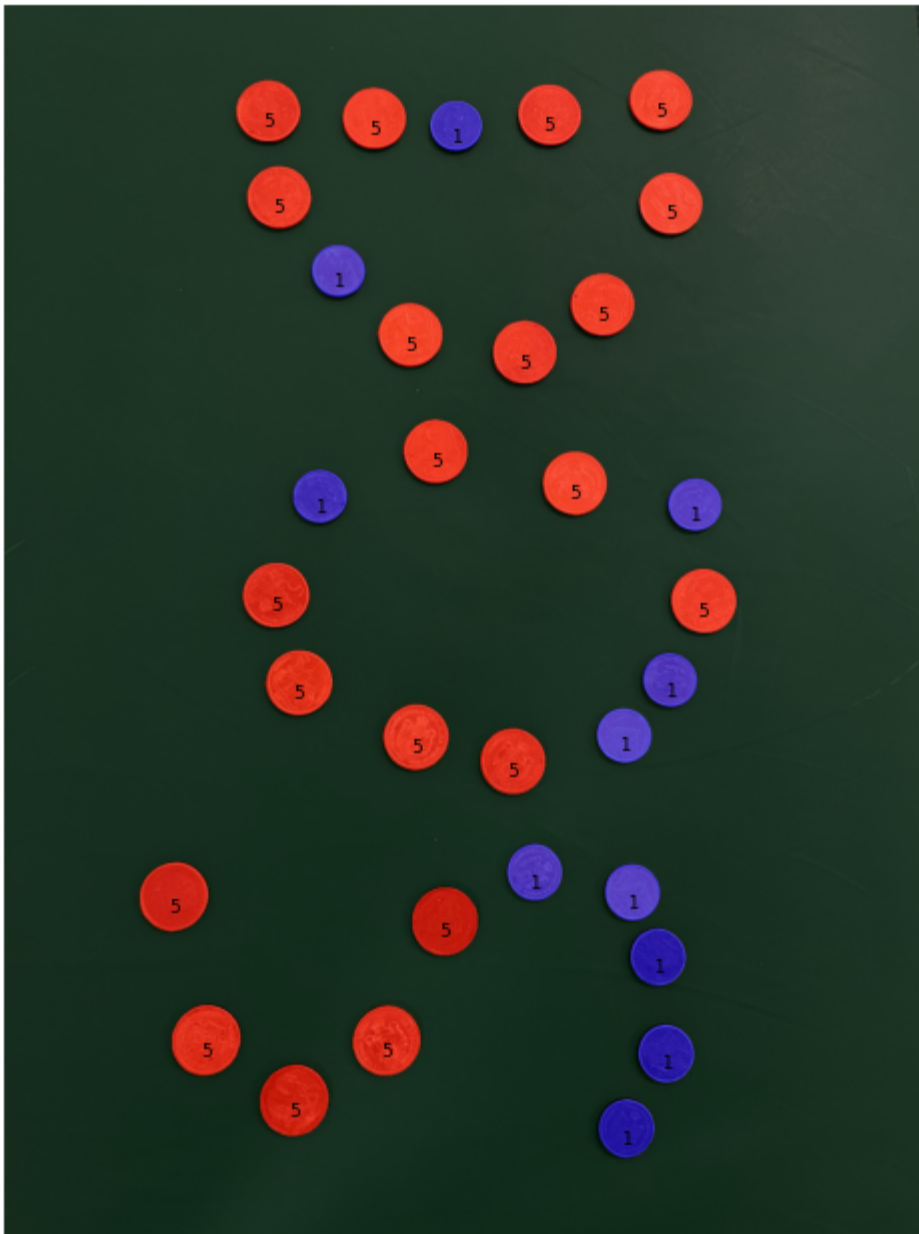


Ulazna slika



labelirana slika

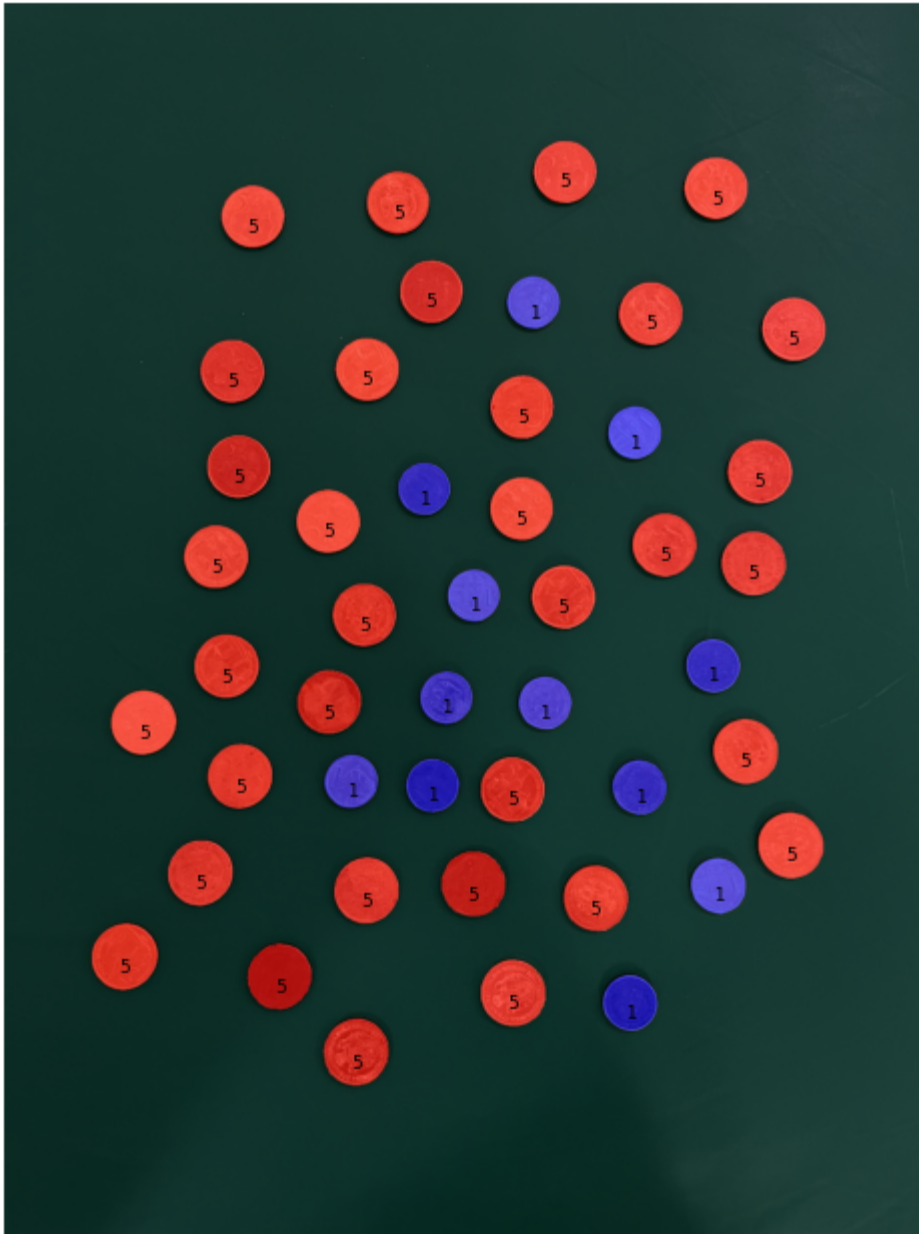


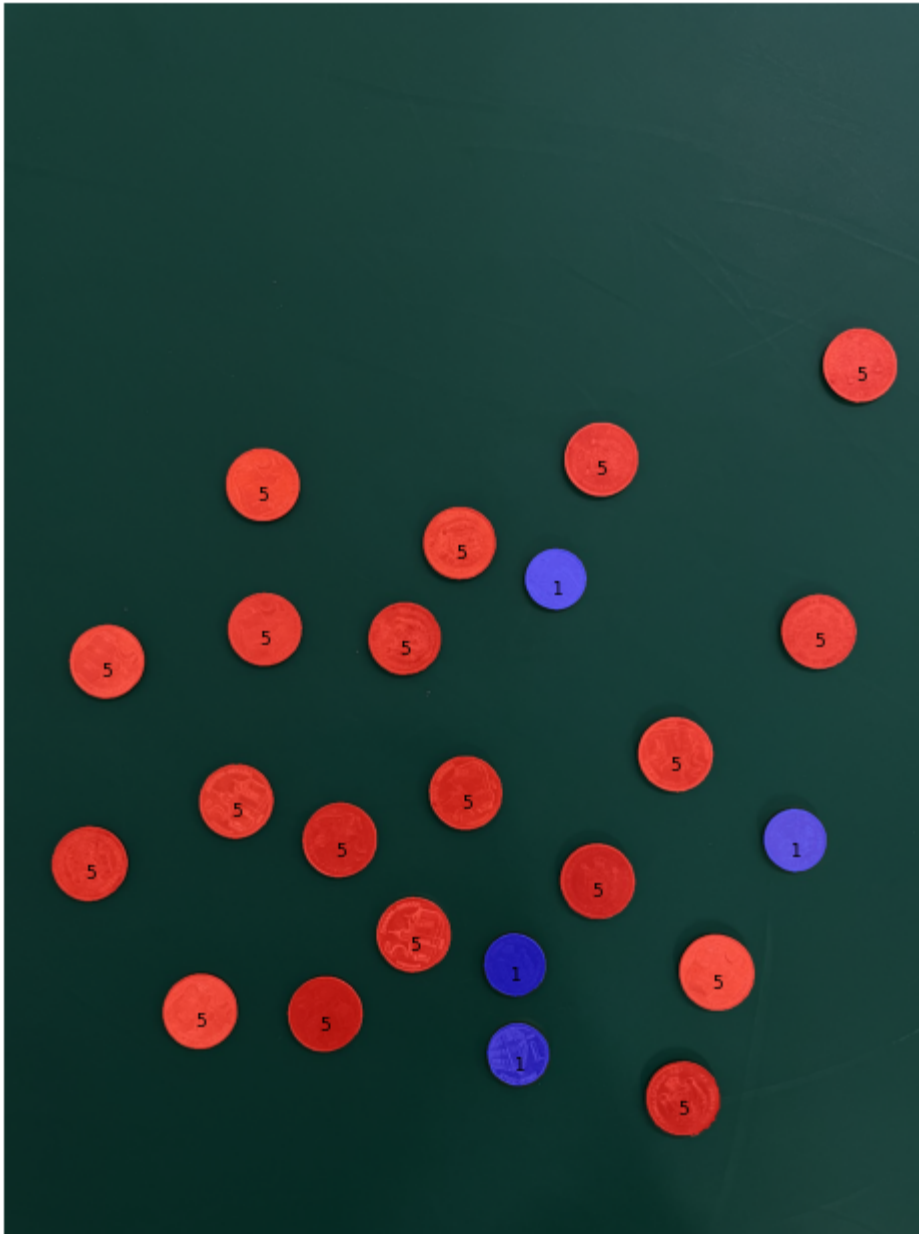


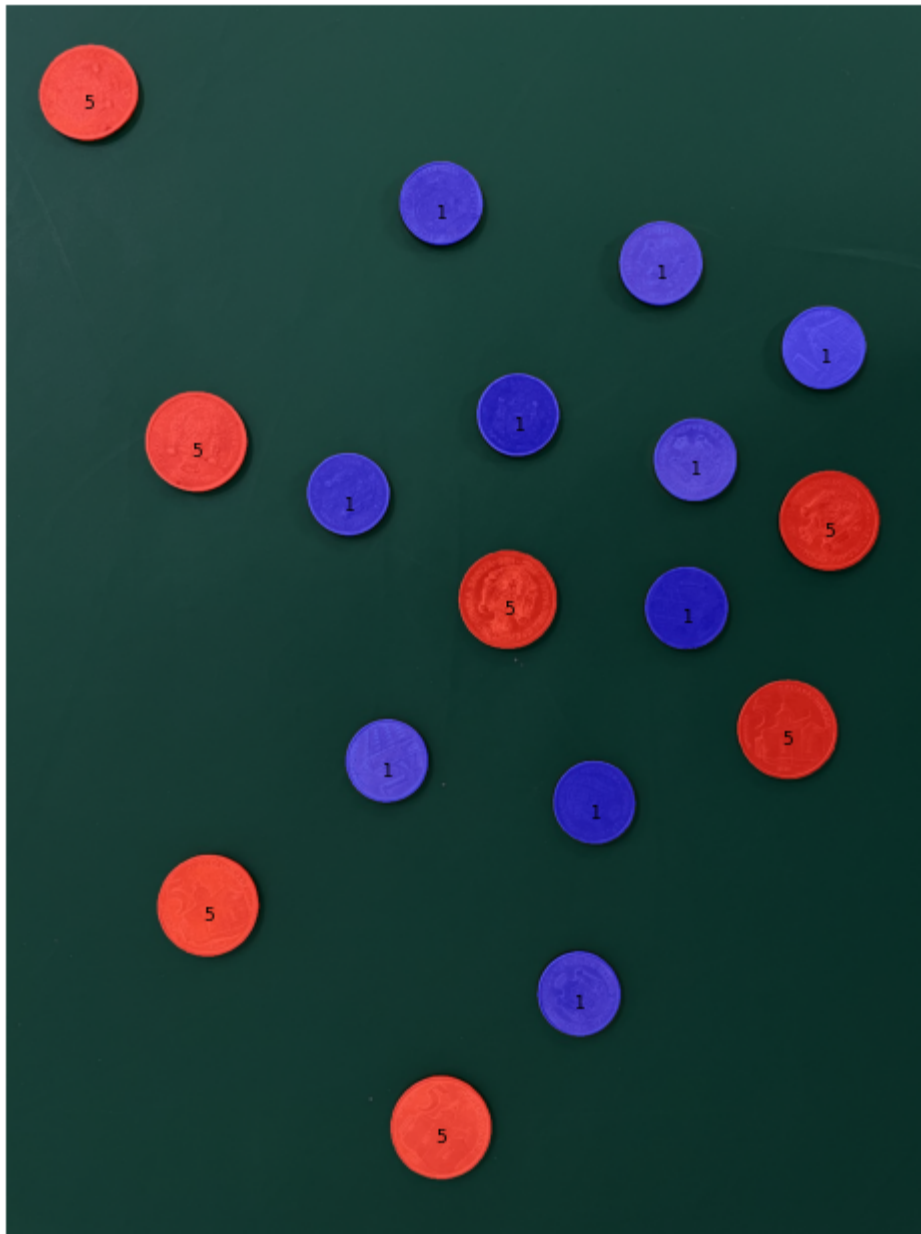
## Izlazni rezultati za sve slike

In [35]:

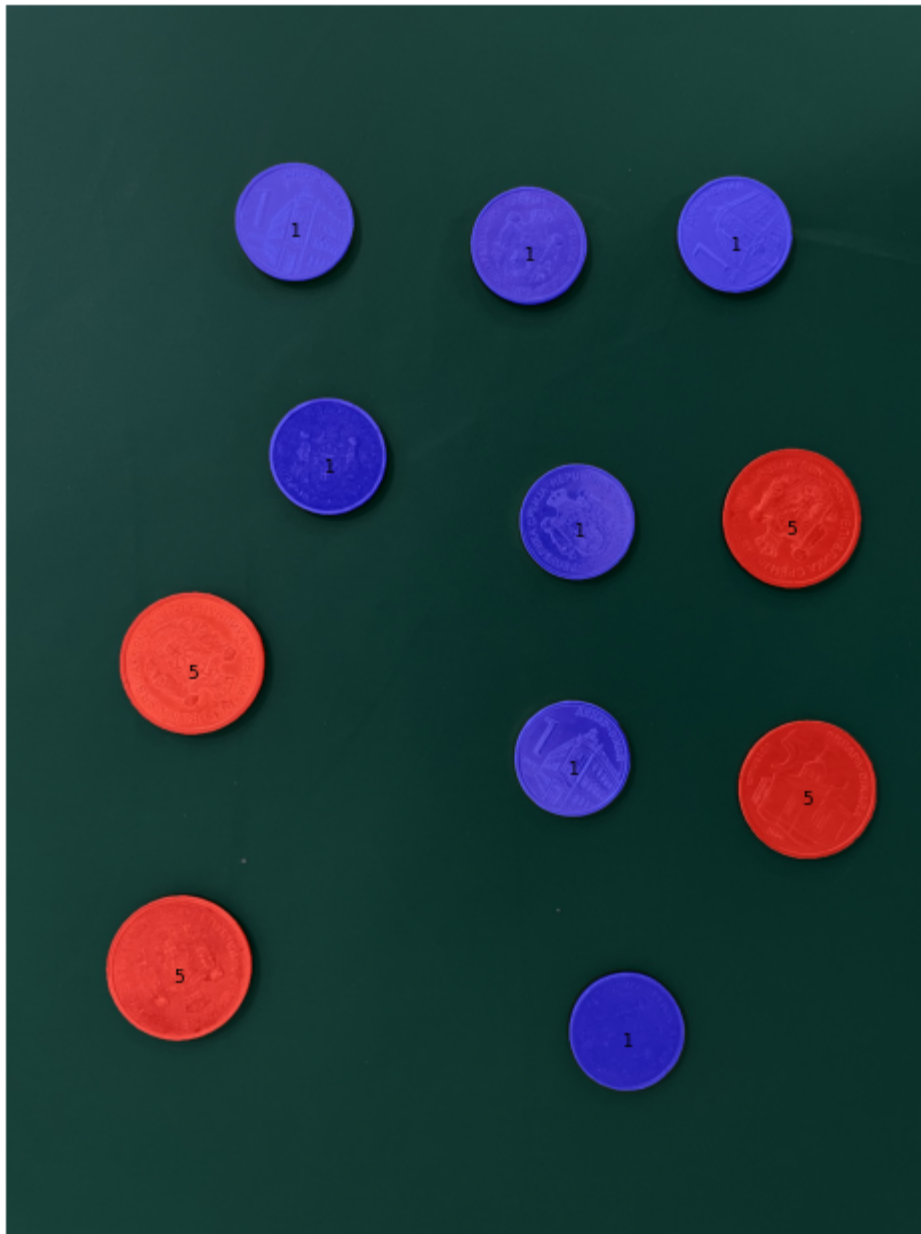
```
output_coins1 = plot_final_coins(image_overlay1, regions1, classes1)
output_coins2 = plot_final_coins(image_overlay2, regions2, classes2)
output_coins3 = plot_final_coins(image_overlay3, regions3, classes3)
output_coins4 = plot_final_coins(image_overlay4, regions4, classes4)
output_coins5 = plot_final_coins(image_overlay5, regions5, classes5)
output_coins6 = plot_final_coins(image_overlay6, regions6, classes6)
output_coins7 = plot_final_coins(image_overlay7, regions7, classes7)
output_coins8 = plot_final_coins(image_overlay8, regions8, classes8)
output_coins9 = plot_final_coins(image_overlay9, regions9, classes9)
```

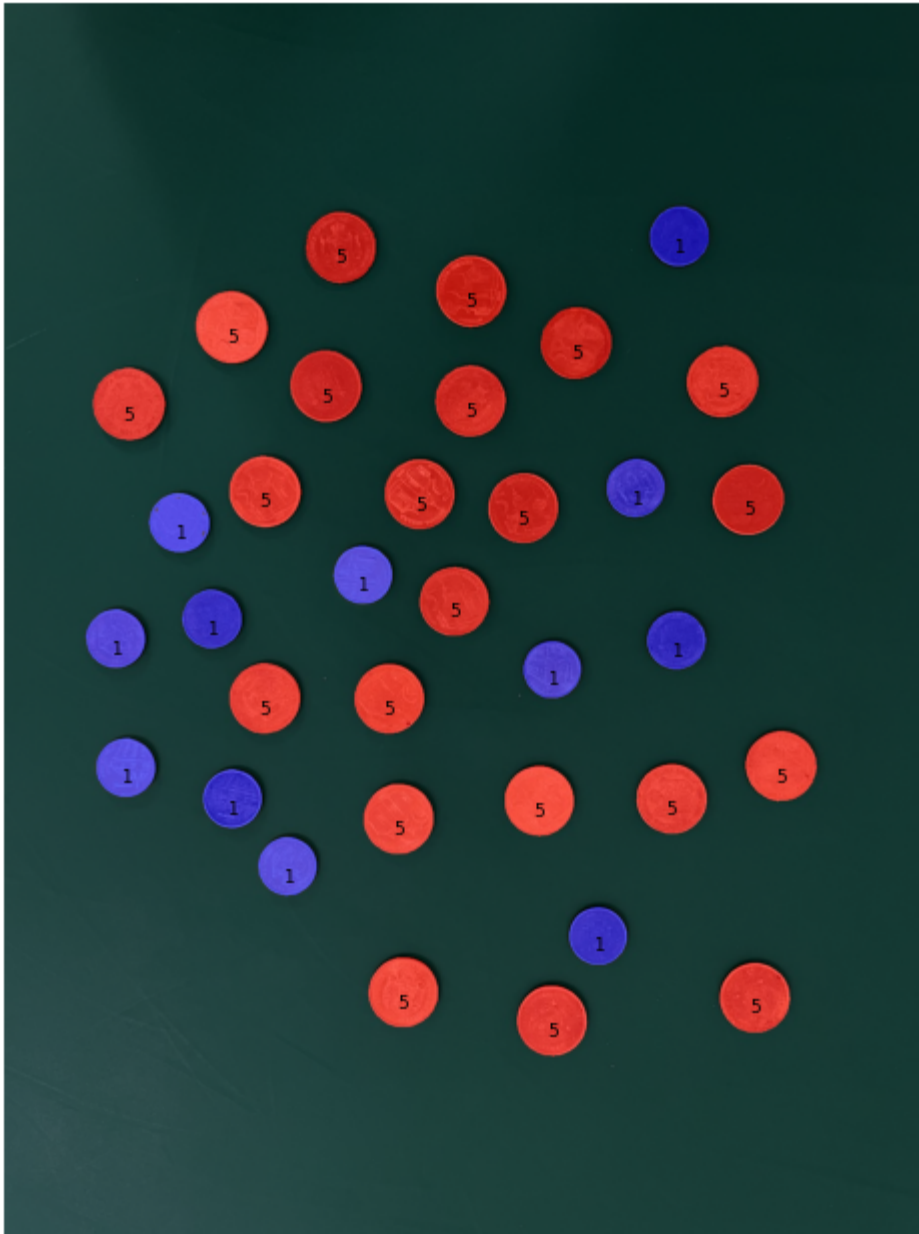


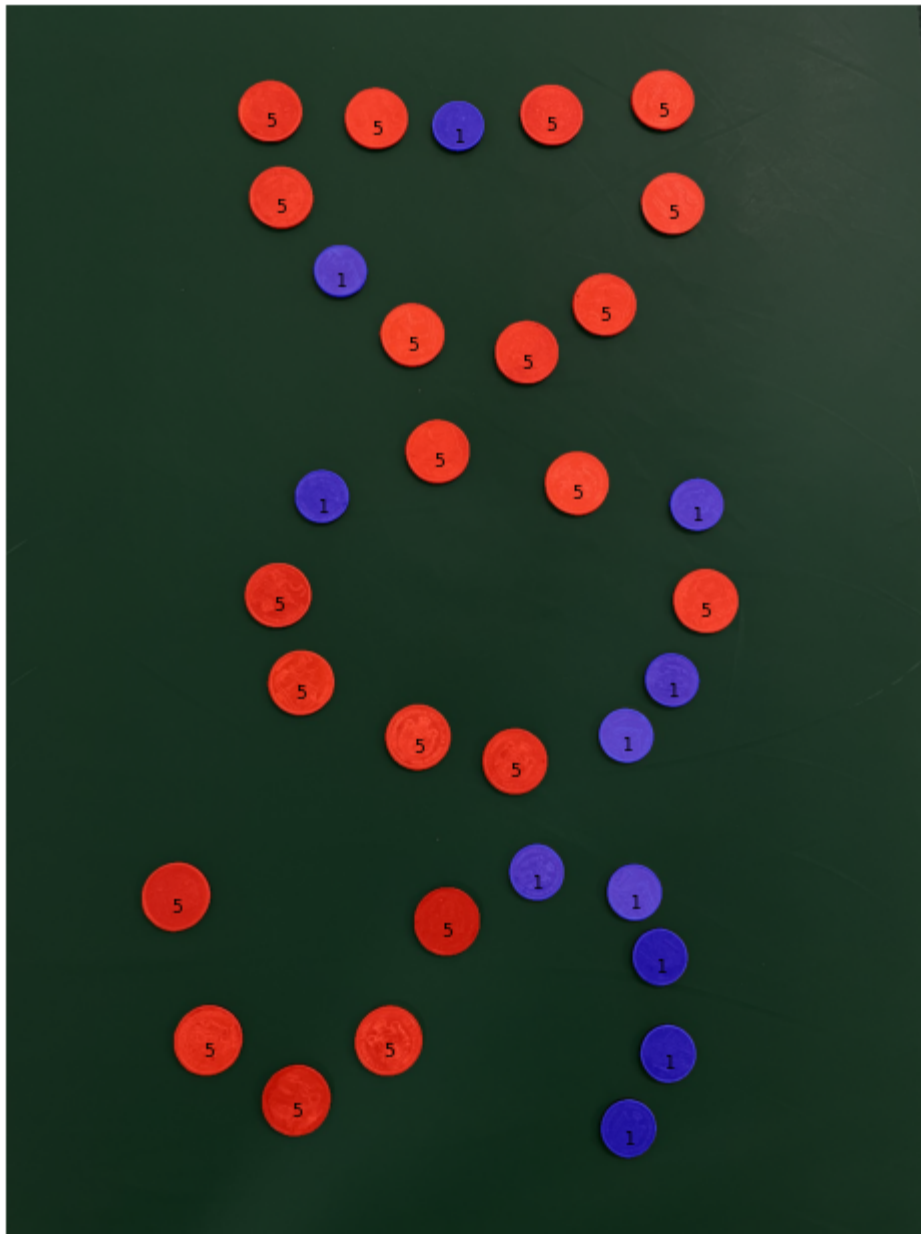


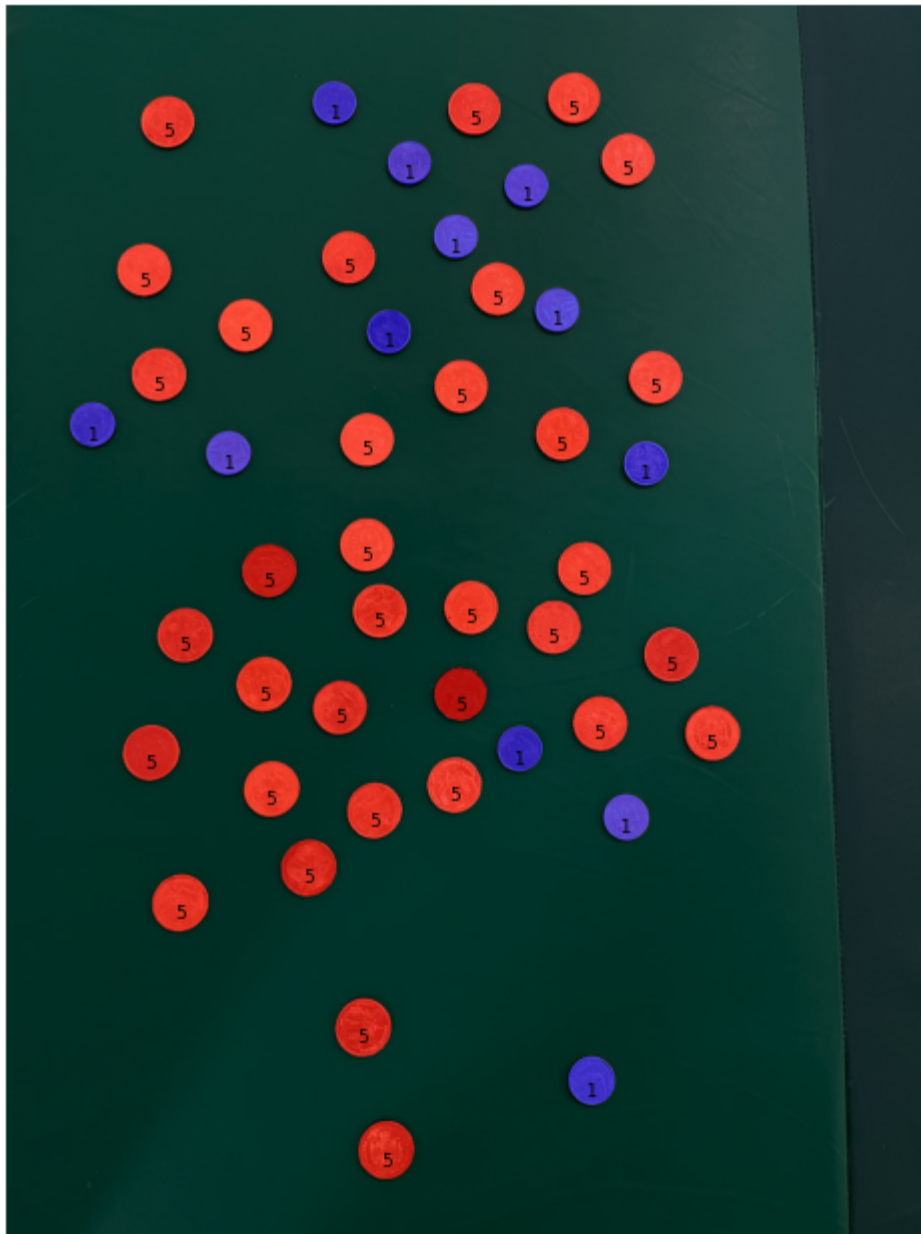


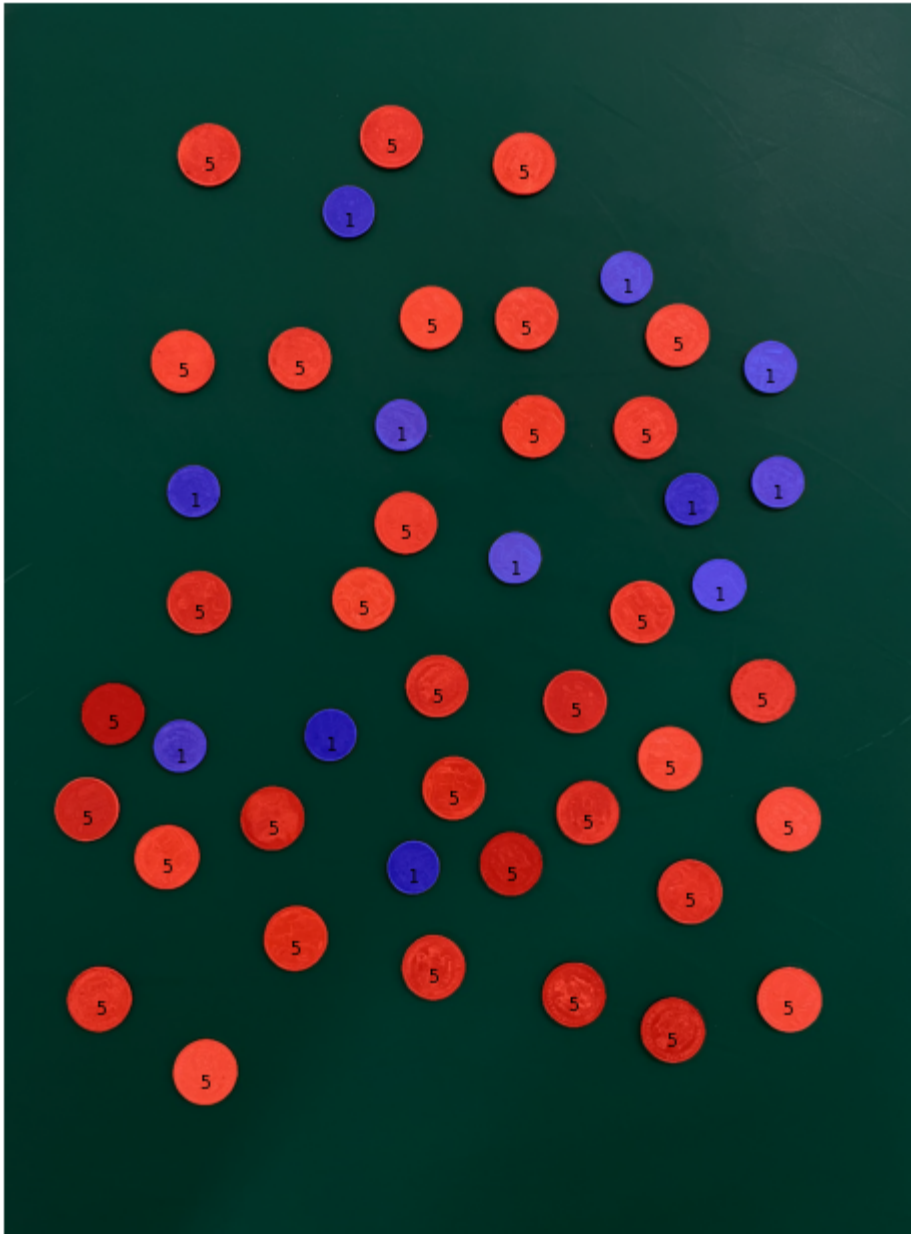


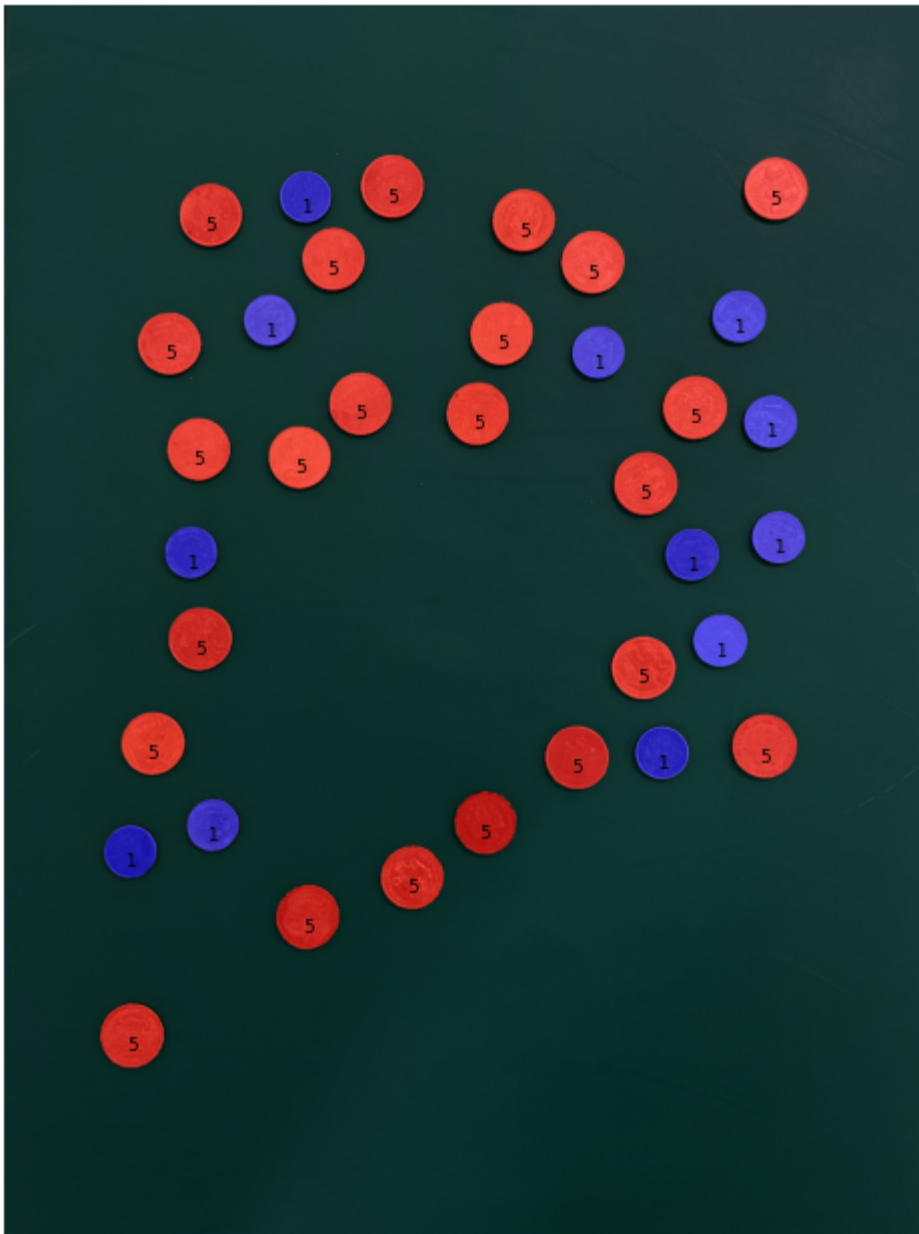












In [36]:

```
# cuvanje izlaznih rezultata slike
ski.io.imwrite("imgCoins1.jpg", output_coins1)
ski.io.imwrite("imgCoins2.jpg", output_coins2)
ski.io.imwrite("imgCoins3.jpg", output_coins3)
ski.io.imwrite("imgCoins4.jpg", output_coins4)
ski.io.imwrite("imgCoins5.jpg", output_coins5)
ski.io.imwrite("imgCoins6.jpg", output_coins6)
ski.io.imwrite("imgCoins7.jpg", output_coins7)
ski.io.imwrite("imgCoins8.jpg", output_coins8)
ski.io.imwrite("imgCoins9.jpg", output_coins9)
```

## 2) DEO

Prikažemo postupak implementacije Canny algoritma na slici **Lena**, korak po korak, uz detaljno objašnjenje svakog koraka.

Nakon toga, isti postupak će biti primenjen na još dve karakteristične slike, kako bismo prikazali univerzalnost algoritma.

Na kraju će biti prikazani svi međurezultati zajedno, što omogućava prikaz svakog koraka: od filtriranja Gausovim filtrom, preko izračunavanja gradijenata i kvantizacije, do potiskivanja lokalnih maksimuma i primene pragova za detekciju jakih i slabih ivica.

```
In [37]: lena_img = ski.util.img_as_float(ski.io.imread("../sekvence/lena.tif"))
```

## Korak 1: Filtriranje slike Gausovim filtrom ( `gauss_filter` )

Prvi korak Kanijevog algoritma je uklanjanje šuma sa ulazne slike korišćenjem **Gausovog filtra**. Funkcija `gauss_filter` vrši ovu operaciju i priprema sliku za dalje korake detekcije ivica.

### Detalji implementacije:

#### 1. Određivanje veličine kernela

- Veličina Gausovog prozora se uzima kao prvi neparni ceo broj veći ili jednak  $6\sigma$ .

#### 2. Kreiranje Gausovog kernela

- Generišu se koordinate X i Y pomoću `np.meshgrid`.
- Računa se Gausova funkcija:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- Kernel se normalizuje tako da suma svih elemenata bude 1.

#### 3. Primena kernela na sliku

- Konvolucija se vrši funkcijom `ndi.convolve` sa režimom "reflect" kako bi se obradile ivice slike.
- Rezultat je **glatka slika** sa smanjenim šumom, spremna za izračunavanje gradijenata u narednim koracima.
- Slika lene nakon primene Gausovog filtra je prikazana nakon što se pokrene ćelija ispod ćelije sa `gaus_filter` funkcijom.



In [109...

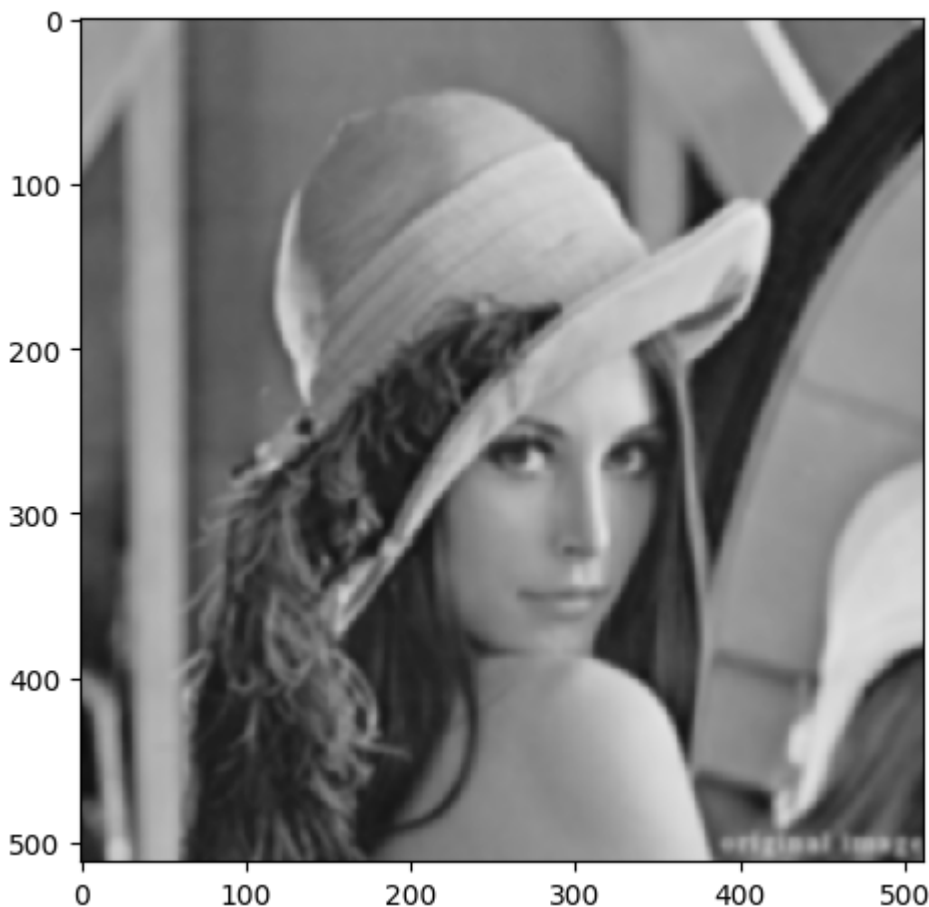
```
def gauss_filter(I, sigma):  
    #ako slika nije [0, 1) prebaci je u [0, 1)  
    windowLen = int(np.ceil(6 * sigma))  
  
    if windowLen % 2 == 0:  
        windowLen += 1  
  
    # pravljenje kernela gaussove funkcije  
    k = windowLen // 2  
    coords = np.arange(-k, k+1)  
    X, Y = np.meshgrid(coords, coords, indexing = 'ij')  
  
    kernel = (1 / (2 * np.pi * sigma **2)) * np.exp(-(X**2 + Y**2)/(2 * sigma**2))  
  
    kernel /= np.sum(kernel)  
  
    I = ndi.convolve(I, kernel, mode = "reflect")  
  
    return I
```

In [107...

```
lena_filtered = gauss_filter(lena_img, 2.0)  
ski.io.imshow(lena_filtered)
```

Out[107...

&lt;matplotlib.image.AxesImage at 0x7f358ec32160&gt;



Funkcija calculateGradients

Funkcija računa horizontalne i vertikalne gradijente slike pomoću **prvog izvoda Gaussove funkcije (DROG operator)**:

1. Prvo se pravi **1D Gauss kernel** i njegov prvi izvod (  $G_{\text{prime}}$  ).
2. Horizontalni gradijent  $G_x$  se dobija **konvolucijom po X osi** sa  $G_{\text{prime}}$  .
3. Vertikalni gradijent (  $G_y$  ) se dobija **konvolucijom po Y osi** sa  $G_{\text{prime}}$  .
4. `convolve1d` sa `mode="reflect"` osigurava da se ivice pravilno obrađuju bez gubitka podataka na granici slike.

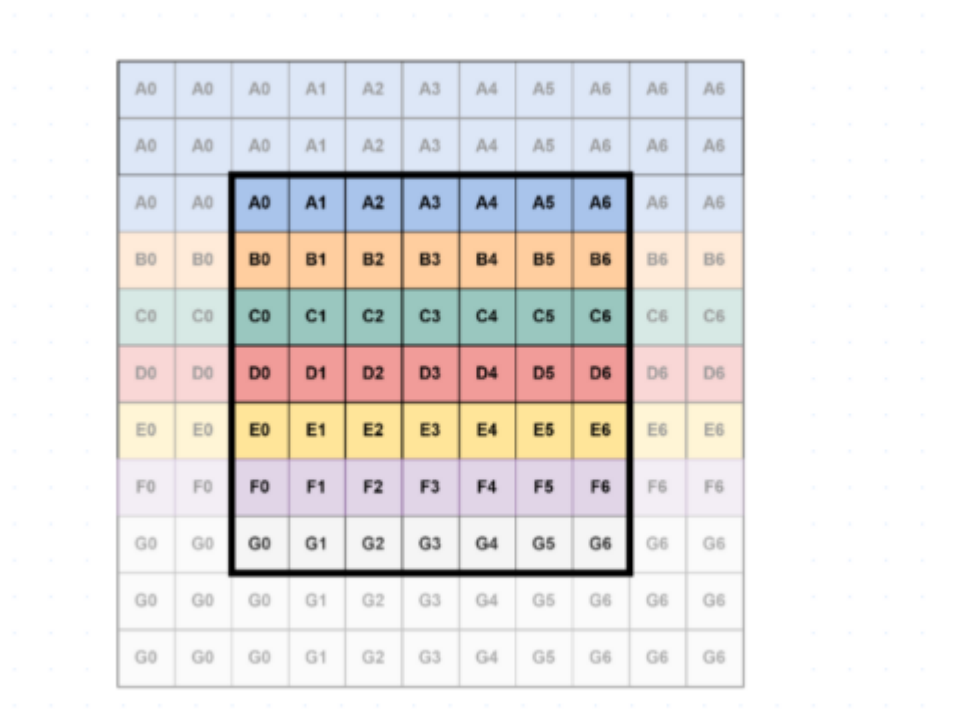
Rezultat su  $G_x$  – gradijent duž horizontalne ose. ,  $G_y$  – gradijent duž vertikalne ose.

Ovi gradijenti se kasnije koriste za računanje magnitude i ugla gradijenta u Kanijevom algoritmu.

Značenje nearest moda ako se koristi prozor 5x5 veličine. Slično tome reflect bi bio zamena pixela kao u ogledalu kad se gleda slika.

In [42]:

```
nearest = ski.io.imread("../sekvence/nearest.jpg")
plt.imshow(nearest)
plt.axis('off')
plt.show()
```



In [124...

```

# def calculateGradients(I, sigma = 2.0):
#     size = int(np.ceil(6 * sigma))
#     if size % 2 == 0:
#         size += 1
#     k = size // 2
#     x = np.arange(-k, k+1)

#     G = np.exp(-x**2 / (2*sigma**2))
#     G /= G.sum()

#     # Prvi izvod Gausa
#     G_prime = -x / (sigma**2) * G
#     G_prime /= np.sum(np.abs(G_prime)) # normalizacija

#     # 2. Horizontalni gradijent
#     # 1D konvolucija po X sa derivatom Gausa
#     Gx = ndi.convolve1d(I, G_prime, axis=1, mode='reflect')

#     # 3. Vertikalni gradijent
#     # 1D konvolucija po Y sa derivatom Gausa
#     Gy = ndi.convolve1d(I, G_prime, axis=0, mode='reflect')

#     return Gx, Gy

def calculateGradients(I):
    Hx = np.array([[ -1, -2, -1],
                   [ 0,  0,  0],
                   [ 1,  2,  1]])
    Hy = np.transpose(Hx)

    Gx = ndi.correlate(I, Hx, mode='nearest')
    Gy = ndi.correlate(I, Hy, mode='nearest')
    return Gx, Gy

```

## Vizualizacija gradijenata

Horizontalni ( Gx\_lena ) i vertikalni ( Gy\_lena ) gradijenti filtrirane slike prikazani su u grayscale-u.

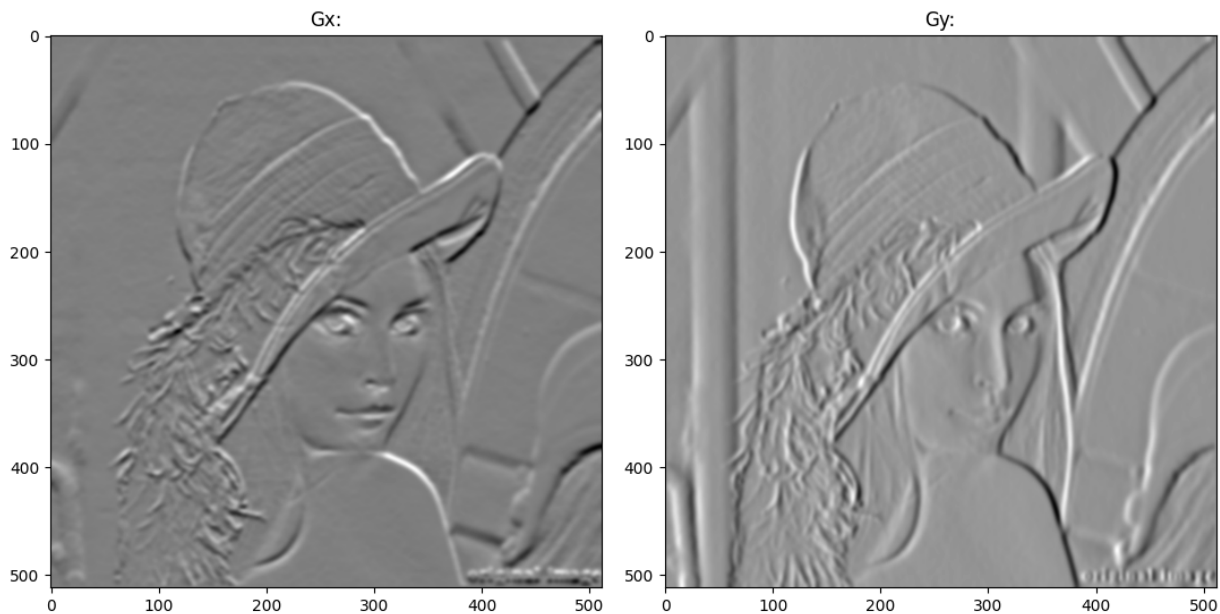
Ove slike pokazuju jačinu promena duž horizontalne i vertikalne ose.

In [125...

```

Gx_lena, Gy_lena = calculateGradients(lena_filtered)
fig, ax = plt.subplots(1, 2, figsize = (11,7), dpi = 100)
ax[0].imshow(Gx_lena, cmap = 'gray', vmin=np.amin(Gx_lena), vmax=np.amax(Gx_lena))
ax[0].set_title("Gx:")
ax[1].imshow(Gy_lena, cmap = 'gray', vmin=np.amin(Gy_lena), vmax=np.amax(Gy_lena))
ax[1].set_title("Gy:")
plt.tight_layout()
plt.show()

```



## Funkcija calculateAngle

Funkcija računa **magnitudu i ugao gradijenta** za ulazne horizontalne i vertikalne gradijente slike  $G_x$  i  $G_y$ .

### Koraci:

#### 1. Magnituda gradijenta:

$$\text{mag} = \sqrt{G_x^2 + G_y^2}$$

- Prikazuje jačinu promena u slici, tj. koliko je intenzitet ivice jak.

#### 2. Ugao gradijenta:

$$\theta = \arctan \frac{G_y}{G_x}$$

- Konvertuje se iz radijana u stepene pomoću `np.rad2deg`.
- Negativni uglovi se pomeraju dodavanjem  $180^\circ$ , tako da bi svi uglovi bili u opsegu  $[0^\circ, 180^\circ]$ .

In [126...

```
def calculateAngle(Gx, Gy):
    mag = np.sqrt(np.square(Gx) + np.square(Gy))
    angle2 = np.rad2deg(np.arctan2(Gy, Gx))

    angle2[angle2 < 0] += 180

    return mag, angle2
```

### Primena na sliku Lena:

- Nakon što su izračunati horizontalni i vertikalni gradijenti ( $G_{x\_lena}$ ,  $G_{y\_lena}$ ) u prošloj

funkciji, trenutna funkcija vraća:

- `mag_lena` -> magnitude gradijenata (prikazuje jačinu ivica).
- `angle_lena` -> ugao gradijenta u stepenima (pokazuje smer ivica).
- Ove dve matrice se mogu prikazati pomoću `imshow`, pri čemu je magnitudeu sivim bojama ( `cmap='gray'` ), a za prikaz ugla je korišćeno `cmap='jet'` sa colorbar-om za intenzitet ugla.

In [127...

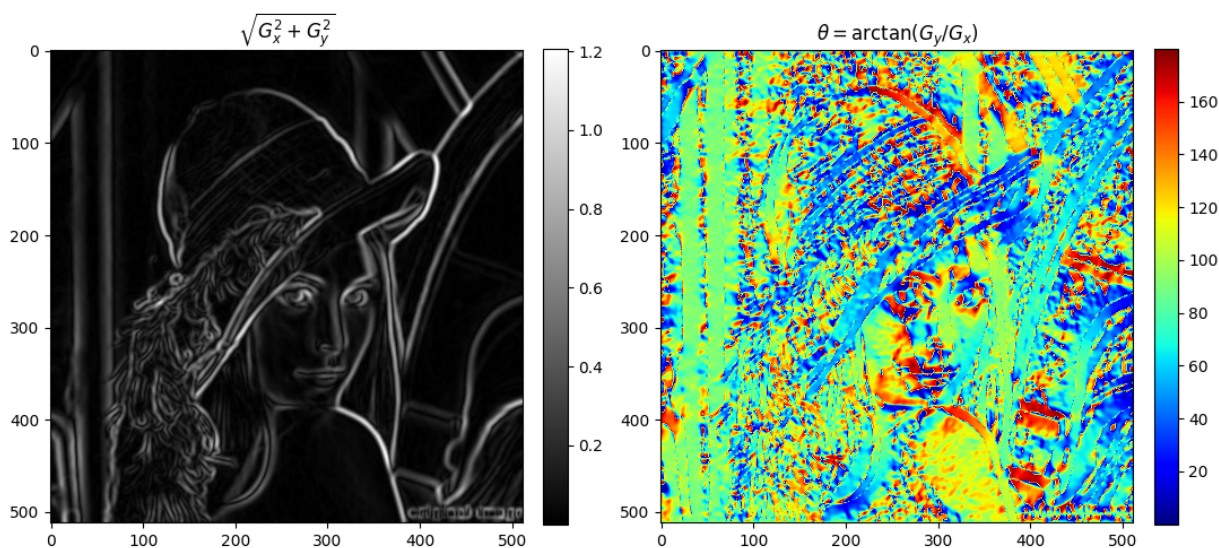
```
mag_lena, angle_lena = calculateAngle(Gx_lena, Gy_lena)

fig, ax = plt.subplots(1, 2, figsize=(11,7), dpi=100)

# Magnitude
im0 = ax[0].imshow(mag_lena, cmap='gray')
ax[0].set_title(r"$\sqrt{G_x^2 + G_y^2}$")
plt.colorbar(im0, ax=ax[0], fraction=0.046, pad=0.04) # dodaje colorbar

# Angle
im1 = ax[1].imshow(angle_lena, cmap='jet')
ax[1].set_title(r"$\theta = \arctan(G_y/G_x)$")
plt.colorbar(im1, ax=ax[1], fraction=0.046, pad=0.04) # colorbar pokazuje vrednosti

plt.tight_layout()
plt.show()
```



## Funkcija `imgQuantization`

Funkcija kvantizuje uglove gradijenta ( `angle2` ) na **četiri osnovna pravca**.

### Koraci:

1. Kreira se matrica iste veličine kao `angle2` popunjena nulama ( `angle_q` ).
2. Uglovi se raspoređuju u četiri kategorije:
  - **0°** → uglovi između 0°–22.5° i 157.5°–180°
  - **45°** → uglovi između 22.5°–67.5°

- $90^\circ \rightarrow$  uglovi između  $67.5^\circ$ – $112.5^\circ$
- $-45^\circ \rightarrow$  uglovi između  $112.5^\circ$ – $157.5^\circ$

3. Rezultat je matrica `angle_q` sa vrednostima koje predstavljaju **kvantizovane pravce gradijenta**.

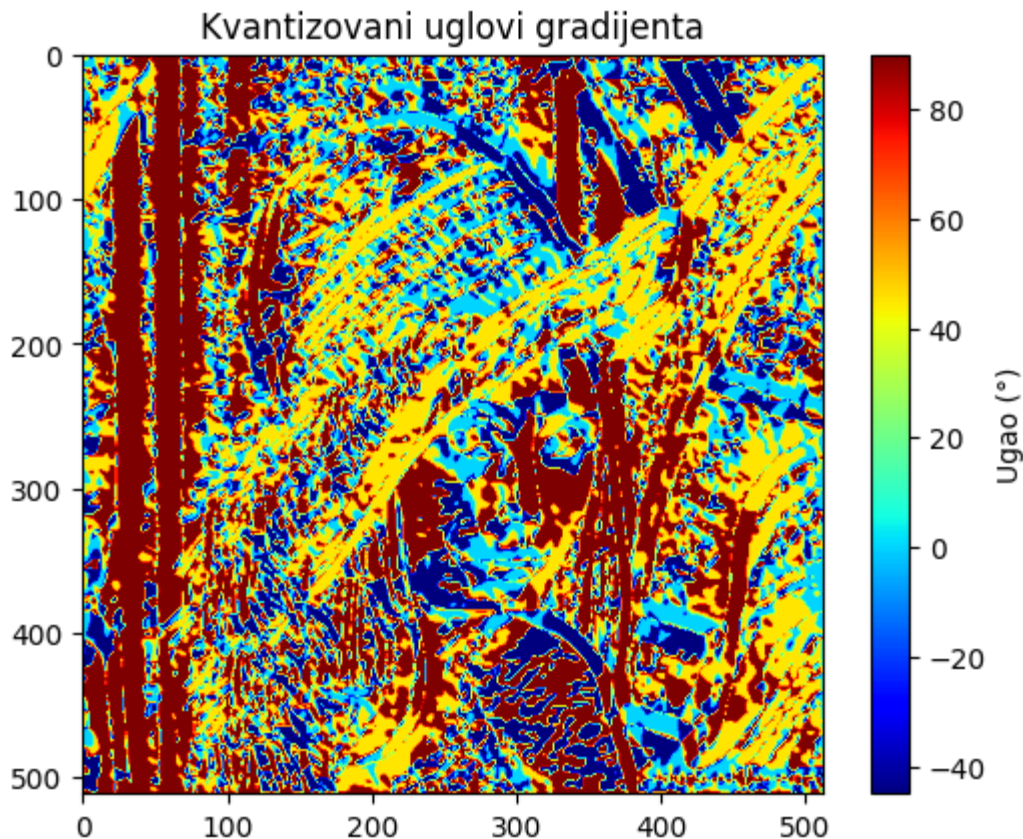
In [128...

```
def imgQuantization(angle2):  
    angle_q = np.zeros_like(angle2)  
  
    #negativni uglovi koji su samo ih pre  
    # 0 stepeni  
    angle_q[(angle2 >= 0) & (angle2 < 22.5)] = 0  
    angle_q[(angle2 >= 157.5) & (angle2 < 180)] = 0  
  
    # 45 stepeni  
    angle_q[(angle2 >= 22.5) & (angle2 < 67.5)] = 45  
  
    # 90 stepeni  
    angle_q[(angle2 >= 67.5) & (angle2 < 112.5)] = 90  
  
    # 135 stepeni  
    angle_q[(angle2 >= 112.5) & (angle2 < 157.5)] = -45  
  
    return angle_q
```

In [129...

```
angle_q_lena = imgQuantization(angle_lena)  
  
# Angle  
plt.imshow(angle_q_lena, cmap='jet')  
plt.title("Kvantizovani uglovi gradijenta")  
plt.colorbar(label="Ugao (°)")  
plt.show()
```





## Funkcija magQuantization

Funkcija vrši potiskivanje ne-maksimuma na magnitudama gradijenta slike, koristeći kvantizovane uglove ( `angle_q` ).

### Koraci funkcije:

1. Dobijaju se dimenzije slike ( `rows` , `cols` ) i kreiraju koordinate `X`, `Y` pomoću `np.meshgrid`.
2. Kreiraju se maske za svaki od kvantizovanih pravaca gradijenta:
  - `mask_0` za  $0^\circ$  (vertikalno),
  - `mask_45` za  $45^\circ$  (dijagonalno levo nagore - desno nadole),
  - `mask_90` za  $90^\circ$  (horizontalno),
  - `mask_minus45` za  $-45^\circ$  (dijagonalno desno nagore - levo nadole).
3. Za svaki pravac se upoređuje magnituda trenutnog piksela sa susednim pikselima duž pravca gradijenta:
  - Ako trenutni piksel nije lokalni maksimum, njegova magnituda se postavlja na 0.
  - Sistem po kom se računa gradijent je negativni deo X-ose koji čini ugao gradijenta od nula stepeni pa se u smeru suprotnom od smera kazaljke na satu povećava respektivno ugao.
  - Primer: za horizontalni gradijent ( $90^\circ$ ) se poredi sa levo i desno; za vertikalni ( $0^\circ$ ) sa gore i dole; za dijagonale sa odgovarajućim susedima.
4. `np.clip` se koristi da indksi ne bi izašli van granica slike.



5. Rezultat je matrica `mag` gde su sve vrednosti koje nisu lokalni maksimumi potisnute na 0.

Ovaj korak Kanijevog algoritma **izdvaja tanke i jasne ivice** tako da samo lokalni maksimumi duž pravca gradijenta ostaju, što omogućava precizno detektovanje ivica.

In [130...

```
def magQuantization(mag, angle_q, I):
    rows, cols = I.shape
    X, Y = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')

    # Maske po pravcima kvantizacije
    mask_0 = (angle_q == 0)
    mask_45 = (angle_q == 45)
    mask_90 = (angle_q == 90)
    mask_minus45 = (angle_q == -45)

    # np.where(condition, return if cond is true, return if cond is false)
    # Horizontalni gradijent (90°): Levo-desno
    mag_left = mag[X, np.clip(Y-1, 0, cols-1)]
    mag_right = mag[X, np.clip(Y+1, 0, cols-1)]
    mag = np.where(mask_90 & ((mag < mag_left) | (mag < mag_right)), 0, mag)

    # Vertikalni gradijent (0°): gore-dole
    mag_up = mag[np.clip(X-1, 0, rows-1), Y]
    mag_down = mag[np.clip(X+1, 0, rows-1), Y]
    mag = np.where(mask_0 & ((mag < mag_up) | (mag < mag_down)), 0, mag)

    # Dijagonalni 45°: gore-Levo \ dole-desno
    mag_up_left = mag[np.clip(X-1, 0, rows-1), np.clip(Y-1, 0, cols-1)]
    mag_down_right = mag[np.clip(X+1, 0, rows-1), np.clip(Y+1, 0, cols-1)]
    mag = np.where(mask_45 & ((mag < mag_up_left) | (mag < mag_down_right)), 0, mag)

    # Dijagonalni -45°: gore-desno / dole-Levo
    mag_up_right = mag[np.clip(X-1, 0, rows-1), np.clip(Y+1, 0, cols-1)]
    mag_down_left = mag[np.clip(X+1, 0, rows-1), np.clip(Y-1, 0, cols-1)]
    mag = np.where(mask_minus45 & ((mag < mag_up_right) | (mag < mag_down_left)), 0,

    return mag
```

## Magnituda posle non-maximum suppression

`lena_new_mag` prikazuje magnitudu gradijenta sa potisnutim ne-maksimumima, što daje tanke i jasne ivice.

Prikazano je u sivim nijansama sa `colorbar` -om.

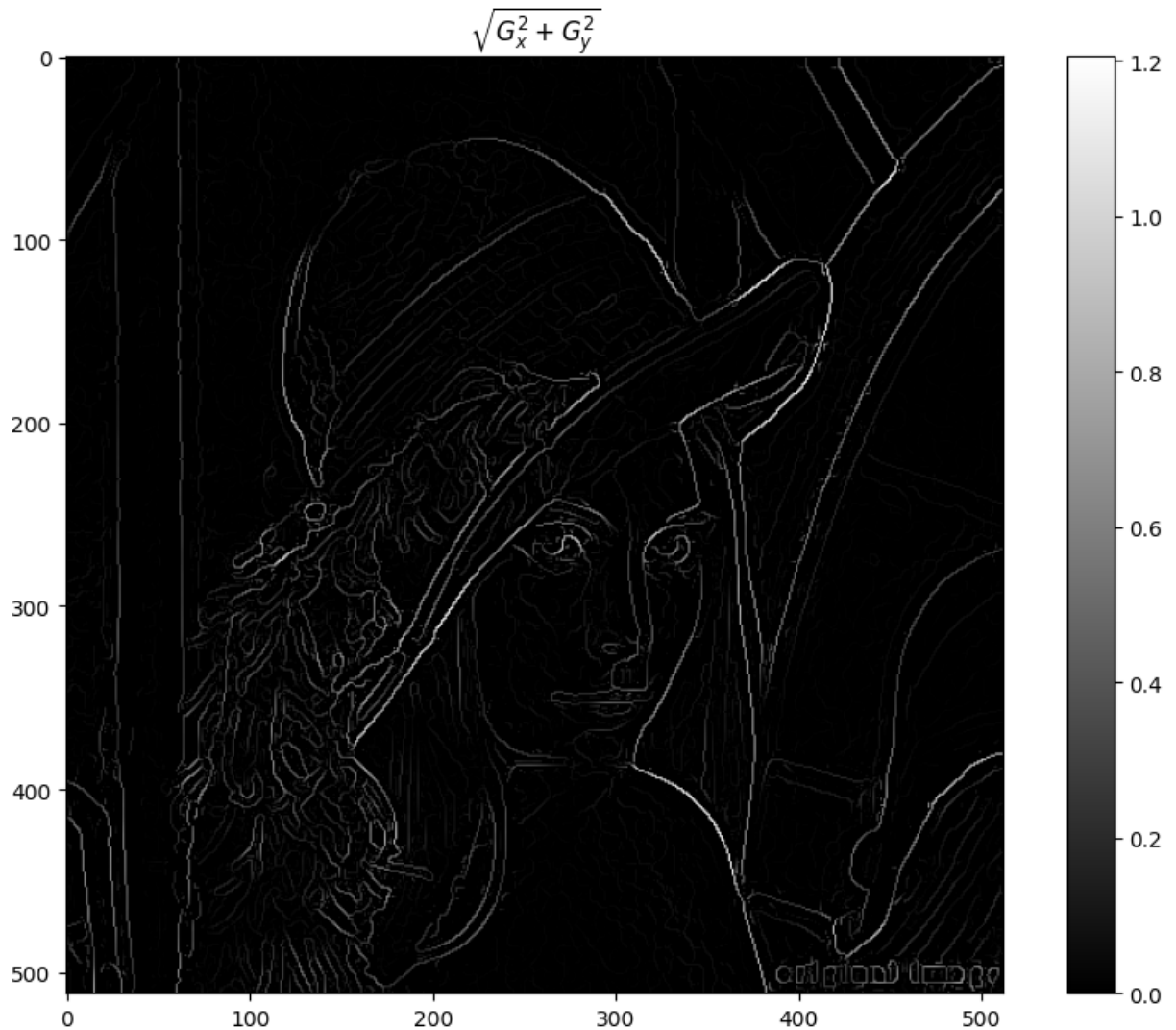
In [131...

```
lena_new_mag = magQuantization(mag_lena, angle_q_lena, lena_filtered)

fig, ax = plt.subplots(1, 1, figsize=(11,7), dpi=100)

# Magnituda nova
im0 = ax.imshow(lena_new_mag, cmap='gray')
ax.set_title(r"$\sqrt{G_x^2 + G_y^2}$")
plt.colorbar(im0, ax=ax, fraction=0.046, pad=0.04) # dodaje colorbar

plt.tight_layout()
plt.show()
```



## Funkcija detectStrongEdges

Funkcija realizuje **histerezis u Canny algoritmu** i služi za povezivanje slabih ivica sa jakim ivicama.

Na početku se formiraju dve binarne slike:

- `imgStrongEdges` – mapa jakih ivica (pikseli iz `upperThresh` )

- `imgWeakEdges` – mapa slabih ivica (piksli iz `lowerThresh` )

Zatim se koristi kernel dimenzija  $3 \times 3$  kako bi se proverilo 8-susedstvo svakog piksela.

U svakoj iteraciji proverava se da li slabi piksel ima bar jednog jakog suseda.

Ako ima, taj slabi piksel se promovise u jaku ivicu.

Proces se ponavlja dok više ne postoji nijedan slabi piksel povezan sa jakim ivicom.

Na kraju funkcija vraća binarnu sliku `imgStrongEdges` koja sadrži samo validne ivice.

Na ovaj način se uklanjaju izolovane slabe ivice, dok se očuvaju kontinualne ivice objekata.

In [132...

```
def detectStrongEdges(mag, lowerThresh, upperThresh):
    imgStrongEdges = np.zeros_like(mag)
    imgWeakEdges = np.zeros_like(mag)

    imgStrongEdges[upperThresh] = 1
    imgWeakEdges[lowerThresh] = 1

    kernel = np.ones((3, 3))

    while True:
        # za svaki piksel: da li u 8-susedstvu postoji jaka ivica
        # piksel ce biti 1 ako u njegovom 8-susedstvu ima jakih ivica
        has_strong_neighbor = ndi.convolve(
            (imgStrongEdges > 0).astype(np.uint8),
            kernel,
            mode='constant'
        ) > 0

        # slabi piksel + ima jakog suseda
        connected = (imgWeakEdges > 0) & has_strong_neighbor

        # ako nema nijednog -> kraj
        if not connected.any():
            break

        # prebaci povezane slabe u jake
        imgStrongEdges[connected] = imgWeakEdges[connected]
        imgWeakEdges[connected] = 0
    return imgStrongEdges
```

In [133...

```
def plotThresholdAndStrongEdgesLena(mag, abs_lower, abs_upper):
    """
    Prikazuje upperMask, lowerMask i strongEdges, vraća strongEdges.
    """
    # 6. Threshold maske
    upperMask = mag > abs_upper
    lowerMask = (mag <= abs_upper) & (mag > abs_lower)

    # 7. Histerezis (strong edges)
    strongEdges = detectStrongEdges(mag, lowerMask, upperMask)

    # Crtanje u tri dela
    fig, ax = plt.subplots(1, 3, figsize=(15, 5))
    ax[0].imshow(upperMask, cmap='gray')
    ax[0].set_title("Gornji prag")
    ax[1].imshow(lowerMask, cmap='gray')
    ax[1].set_title("Donji prag")
    ax[2].imshow(strongEdges, cmap='gray')
    ax[2].set_title("Rezultat")
    plt.tight_layout()
    plt.show()

    return strongEdges
```

Ovde je ostavljen `widgets.interact` da bi se vizualno pokazalo kako promena parametara utiče na rezultate.

Funkcija `plotThresholdAndStrongEdgesLena` koristi fiksnu vrednost `sigma = 2.0`, dok se `abs_lower` i `abs_upper` mogu menjati preko slidera.

Promenom `abs_lower` reguliše se **donja granica za detekciju ivica**, pa se manje ili više slabih ivica uključuje u rezultat.

Promenom `abs_upper` menja se **gornja granica**, što utiče na to koje jake ivice ostaju vidljive.

Interaktivno menjanje ovih parametara omogućava brzo uočavanje efekta praga na konačnu Canny detekciju.

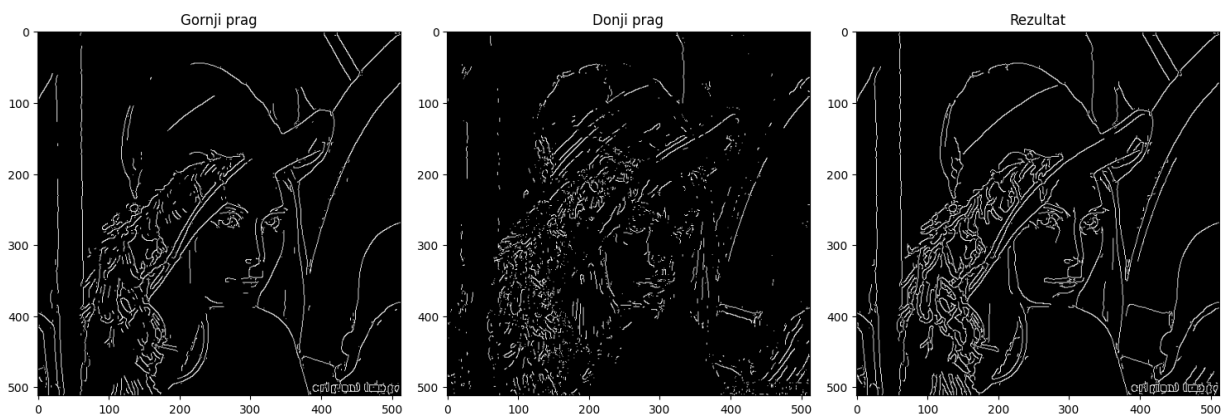
In [ ]:

```
# Lena_final = widgets.interact(plotThresholdAndStrongEdgesLena, mag = fixed(Lena_new
#                               abs_upper = (0, 1, 0.01))

# Konacni parametri abs_lower = 0.09, abs_upper = 0.23
```

In [144...

```
# Poziv funkcije sa fiksnim pragovima
plotThresholdAndStrongEdgesLena(mag=lena_new_mag, abs_lower=0.09, abs_upper=0.23)
```



```
Out[144...] array([[0., 0., 0., ..., 1., 0., 0.],
       [0., 0., 0., ..., 0., 1., 1.],
       [0., 0., 0., ..., 0., 1., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

## Opis funkcija za Canny detekciju ivica

### 1. `calc_strong_edges(mag, abs_lower, abs_upper)`

Ova funkcija implementira **histerezis** Canny algoritma:

- Prvo se kreiraju dve maske:
  - `upperMask` – pikseli gde je magnituda veća od `abs_upper` (jake ivice)
  - `lowerMask` – pikseli gde je magnituda između `abs_lower` i `abs_upper` (slabe ivice)
- Zatim se poziva funkcija `detectStrongEdges` koja povezuje slabe ivice sa jakim, tako da se očuvaju samo kontinuirane ivice.
- Vraća se binarna mapa `strongEdges` koja sadrži samo validne ivice.

### 2. `canny_comparison(image, lena_mag, sigma=2.0, lt=0.09, ht=0.2)`

Ova funkcija služi za **vizuelno poređenje klasične Canny funkcije i implementacije ručno napisane**:

- Poziva `ski.feature.canny` sa parametrima `sigma`, `low_threshold=lt` i `high_threshold=ht`.
- Poziva funkciju `calc_strong_edges` koristeći `lena_mag` i iste pragove.
- Prikazuje rezultate u dva panela:
  - Levo: Canny ugrađena funkcija
  - Desno: Naša implementacija sa histerezom
- Omogućava lako poređenje i evaluaciju rezultata.

### 3. `widgets.interact(...)`

- Koristi se za **interaktivnu kontrolu parametara** Canny algoritma u Jupyter okruženju.
- Parametri poput `sigma`, `lt` (niži prag) i `ht` (viši prag) mogu se menjati pomoću slidera.
- Funkcija `canny_comparison` se poziva automatski kada korisnik menja vrednosti, a prikaz se osvežava.
- Omogućava brzo eksperimentisanje sa različitim parametrima i vizuelnu procenu efekata na detekciju ivica.

In [137...

```
def calc_strong_edges(mag, abs_lower, abs_upper):
    """
    vraća strongEdges na osnovu upper and lower maske.
    """
    # 6. Threshold maske
    upperMask = mag > abs_upper
    lowerMask = (mag <= abs_upper) & (mag > abs_lower)

    # 7. Histerezis (strong edges)
    strongEdges = detectStrongEdges(mag, lowerMask, upperMask)

    return strongEdges
```

In [138...

```
def canny_comparison(image, lena_mag, sigma=2.0, lt=0.09, ht=0.2):
    #image = ski.util.img_as_float(image)
    result = ski.feature.canny(image, sigma=sigma, low_threshold=lt, high_threshold=ht)
    lena_final = calc_strong_edges(lena_mag, lt, ht)

    figure, ax = plt.subplots(nrows=1, ncols=2, figsize=(15, 10), dpi=80)
    ax[0].imshow(lena_final, cmap='gray', vmin=0, vmax=1), ax[0].axis('off'), ax[0].set_title('Original Image')
    ax[1].imshow(result, cmap='gray'), ax[1].axis('off'), ax[1].set_title('Canny ugradnja')
    plt.tight_layout()
    plt.show()
    return
```

In [ ]:

```
# widgets.interact(canny_comparison, image = widgets.fixed(lena_img), lena_mag = widgets.fixed(lena_mag),
#                  sigma = widgets.fixed(2.0), lt = (0, 1, 0.01), ht = (0, 1, 0.01))
```

In [145...

```
canny_comparison(image=lena_img, lena_mag=lena_new_mag, sigma=2.0, lt=0.09, ht=0.23)
```



## visualizeProcessingStages

Funkcija `visualizeProcessingStages` služi za **prikaz svih ključnih koraka Canny detekcije ivica**.

Kreira **3x3 mrežu subplotova** koristeći Matplotlib, tako da se svaki korak vizualizuje zasebno.

**Prvi red** prikazuje:

- Originalnu sliku ( `I` )
- Gaussian-filtered sliku ( `imgGauss` )
- Horizontalne gradijente ( `Gx` )

**Drugi red** prikazuje:

- Vertikalne gradijente ( `Gy` )
- Magnitudu gradijenta ( `mag` )
- Kvantizovani ugao ( `angle_q` ) sa `jet` opcijom

**Treći red** prikazuje:

- Masku jakih ivica ( `upperThresh` )
- Masku slabih ivica ( `lowerThresh` )
- Konačne ivice posle histerezis thresholdinga ( `imgStrongEdges` )

Sve slike imaju isključene ose ( `axis('off')` ) radi lepšeg prikaza, a `plt.tight_layout()` osigurava da se subplotovi ne preklapaju. Na kraju se koristi `plt.show()` za prikaz figure.



In [140...

```

def visualizeProcessingStages(I, imgGauss, Gx, Gy, mag, angle_q,
    upperThresh, lowerThresh, imgStrongEdges):
    fig, axes = plt.subplots(nrows = 3, ncols = 3, figsize = (12, 8), dpi = 100)

    axes[0, 0].imshow(I, cmap='gray')
    axes[0, 0].set_title("Gaussian filtered")
    axes[0, 0].axis('off')

    axes[0, 1].imshow(imgGauss, cmap='gray')
    axes[0, 1].set_title("Gaussian filtered")
    axes[0, 1].axis('off')

    axes[0, 2].imshow(Gx, cmap='gray')
    axes[0, 2].set_title("Gx")
    axes[0, 2].axis('off')

    axes[1, 0].imshow(Gy, cmap='gray')
    axes[1, 0].set_title("Gy")
    axes[1, 0].axis('off')

    axes[1, 1].imshow(mag, cmap='gray')
    axes[1, 1].set_title("Gradient magnitude")
    axes[1, 1].axis('off')

    axes[1, 2].imshow(angle_q, cmap='jet')
    axes[1, 2].set_title("Quantized angle")
    axes[1, 2].axis('off')

    axes[2, 0].imshow(upperThresh, cmap='gray')
    axes[2, 0].set_title("Strong edges mask")
    axes[2, 0].axis('off')

    axes[2, 1].imshow(lowerThresh, cmap='gray')
    axes[2, 1].set_title("Weak edges mask")
    axes[2, 1].axis('off')

    axes[2, 2].imshow(imgStrongEdges, cmap='gray')
    axes[2, 2].set_title("Final edges (hysteresis)")
    axes[2, 2].axis('off')

    plt.tight_layout()
    plt.show()

```

## canny\_edge\_detection

Funkcija `canny_edge_detection` implementira **Canny algoritam za detekciju ivica** kroz nekoliko koraka:

1. **Tip slike:** Ako je ulaz `I` tipa `uint8`, konvertuje se u `float` radi preciznijih proračuna.
2. **Gaussian filter:** Primena Gaussovog filtera ( `gauss_filter` ) za uklanjanje šuma i glatkoću slike.
3. **Gradijenti:** Izračunavaju se horizontalni ( `Gx` ) i vertikalni ( `Gy` ) gradijenti slike ( `calculateGradients` ).

4. **Magnituda i ugao:** Na osnovu gradijenata računa se magnituda i smer gradijenta ( `calculateAngle` ).
5. **Kvantizacija uglova:** Smerovi gradijenata se kvantizuju u diskretne uglove ( `imgQuantization` ).
6. **Non-maximum suppression:** Eliminacija slabih pikselnih lokalnih maksimuma radi tankih ivica ( `magQuantization` ).
7. **Threshold maske:** Formiraju se maske jakih ( `upperThresh` ) i slabih ( `lowerThresh` ) ivica na osnovu apsolutnih pragova.
8. **Histerezis:** Slabe ivice se uključuju samo ako su povezane sa jakim ivicama ( `detectStrongEdges` ).
9. **Vizualizacija:** Prikazuju se svi koraci obrade pomoću funkcije `visualizeProcessingStages` .

Ova funkcija kombinuje sve korake Canny algoritma koji su prethodno objašnjeni na slici lene.

In [141...

```
def canny_edge_detection(I, sigma, abs_lower, abs_upper, visualize = True):
    if I.dtype == np.uint8:
        I = ski.util.img_as_float(I)

    # 1. Gauss filter
    imgGauss = gauss_filter(I, sigma)

    # 2. Gradijenti
    Gx, Gy = calculateGradients(imgGauss)

    # 3. Magnituda i ugao
    mag, angle2 = calculateAngle(Gx, Gy)

    # 4. Kvantizacija uglova
    angle_q = imgQuantization(angle2)

    # 5. Non-maximum suppression
    mag = magQuantization(mag, angle_q, I)

    # 6. Threshold maske
    upperThresh = mag > abs_upper
    lowerThresh = (mag <= abs_upper) & (mag > abs_lower)

    # 7. Histerezis
    imgStrongEdges = detectStrongEdges(mag, lowerThresh, upperThresh)

    # 8. Vizualizacija
    if visualize == True:
        visualizeProcessingStages(I, imgGauss, Gx, Gy, mag, angle_q, upperThresh, lowerThresh)

    return imgStrongEdges
```

## 1. Interaktivna vizualizacija Canny detekcije

U prvoj ćeliji se učitavaju tri slike ( `lena.tif` , `van.tif` , `cameraman.tif` ) koristeći

`skimage.io.imread` .

Zatim se koristi `widgets.interact` da se omogući **interaktivno podešavanje parametara** za funkciju `canny_edge_detection` .

- Parametri koji se mogu menjati: `sigma` (za Gauss filter), `abs_lower` i `abs_upper` (apsolutni pragovi za histerezis).
- Svaka slika se povezuje sa zasebnim `interact` pozivom, tako da se može testirati detekcija ivica nezavisno na svakoj slici.

## 2. Funkcija `canny_usage`

Ćelija definiše funkciju `canny_usage` koja koristi **ugrađeni Canny algoritam** iz `skimage.feature` .

- Ulazni parametri su: `image` (slika), `sigma` (Gauss filter), `lt` (low threshold) i `ht` (high threshold).
- Slika se konvertuje u `float` tip ( `img_as_float` ) radi preciznosti.
- Funkcija poziva `ski.feature.canny` da izvrši detekciju ivica.
- Rezultat i originalna slika se prikazuju u **jednoj figuri sa dva subplot-a**: originalna slika levo, rezultat Canny algoritma desno.
- Obe slike imaju isključene ose ( `axis('off')` ) i koristi se `plt.tight_layout()` da se subplotovi lepo rasporede.

## 3. Pozivanje `canny_usage` na više slika

U trećoj ćeliji se ponovo učitavaju iste tri slike ( `lena` , `van` , `cameraman` ).

Svaka slika se prosleđuje funkciji `canny_usage` , čime se vrši **detekcija ivica koristeći ugrađeni Canny algoritam**.

Rezultat je vizualno poređenje originalne slike i ivica za svaku od tri slike.

In [ ]:

```
img1 = ski.io.imread("../sekvenca/lena.tif")
img2 = ski.io.imread("../sekvenca/van.tif")
img3 = ski.io.imread("../sekvenca/cameraman.tif")

# widgets.interact(canny_usage, I = widgets.fixed(img1), sigma = (0.5, 3), c
#                               abs_upper = (0, 1, 0.01), visualize = widgets.fixed(True))

# widgets.interact(canny_usage, I = widgets.fixed(img2), sigma = (0.5, 3), c
#                               abs_upper = (0, 1, 0.01), visualize = widgets.fixed(True))

# widgets.interact(canny_usage, I = widgets.fixed(img3), sigma = (0.5, 3), c
#                               abs_upper = (0, 1, 0.01), visualize = widgets.fixed(True))

# sigma = 2.00 , abs_lower = 0.09, abs_upper = 0.23 medjukoraci prikazani i widgets.1
```

In [143...

```

# Za img1
edges1 = canny_edge_detection(I=img1, sigma=2.0, abs_lower=0.09, abs_upper=0.23, visu

# Za img2
edges2 = canny_edge_detection(I=img2, sigma=2.0, abs_lower=0.09, abs_upper=0.23, visu

# Za img3
edges3 = canny_edge_detection(I=img3, sigma=2.0, abs_lower=0.09, abs_upper=0.23, visu

# plt.show()

```

Gaussian filtered



Gaussian filtered



Gx



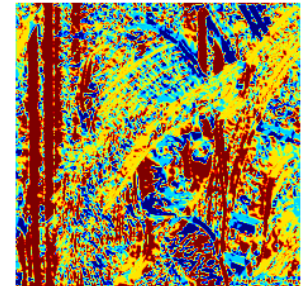
Gy



Gradient magnitude



Quantized angle



Strong edges mask



Weak edges mask



Final edges (hysteresis)



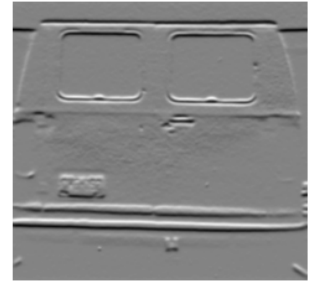
Gaussian filtered



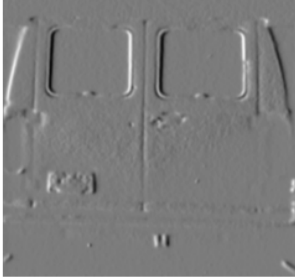
Gaussian filtered



Gx



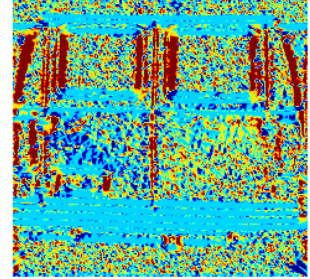
Gy



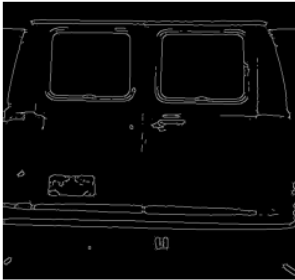
Gradient magnitude



Quantized angle



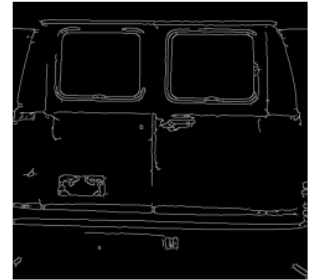
Strong edges mask

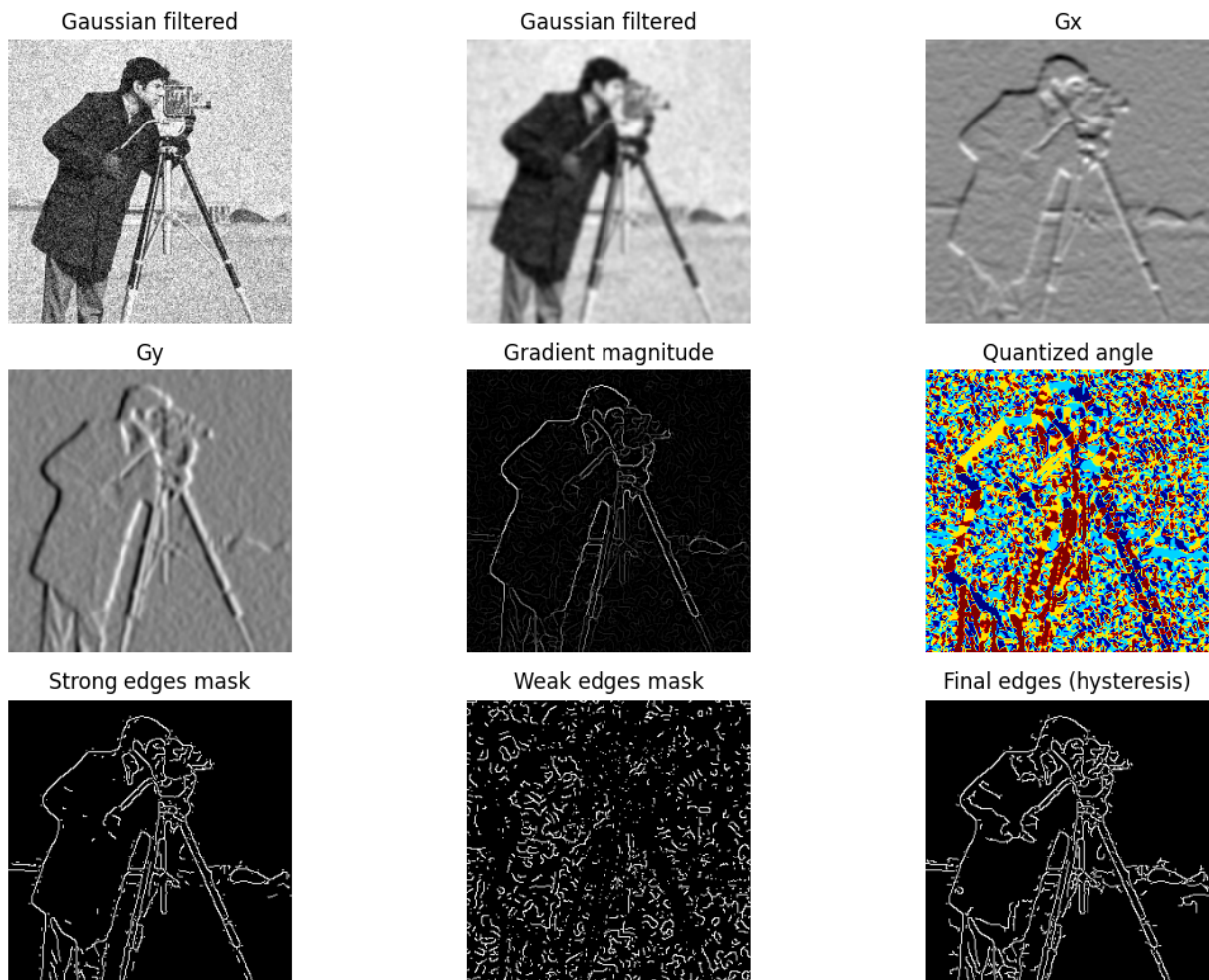


Weak edges mask



Final edges (hysteresis)





Odabrani su sledeći parametri:

- $\sigma = 2.0$
- $\text{abs\_lower} = 0.09$
- $\text{abs\_upper} = 0.23$

## Kriterijum izbora

### 1. Sigma (Gauss filter)

- Određuje glatkoću slike pre detekcije ivica.
- Veća vrednost sigma -> veća glatkoća, manje šuma, ali tanje ivice mogu biti izgubljene.
- Manja vrednost sigma -> više detalja, ali više šuma -> lažne ivice.
- Izabrana vrednost  $\sigma=2.0$  predstavlja kompromis: dovoljno glatko da se eliminiše šum, a da se ne izgube značajne ivice na slikama.

### 2. Apsolutni pragovi (abs\_lower i abs\_upper)

- $\text{abs\_lower}$  filtrira slabije ivice -> previše nisko može detektovati šum.
- $\text{abs\_upper}$  određuje koje ivice se uzimaju kao sigurne -> više visoko može propustiti prave ivice.



- Odabrano `abs_lower=0.09` i `abs_upper=0.23` jer daju balans između očuvanja glavnih ivica i eliminacije šuma.

## Efekat promena parametara

- **Smanjenje sigma (npr. `sigma=1.0`)**
  - Više detalja i fragmenata -> ivice objekata mogu biti previše fragmentovane.
  - Može se javiti više lažnih ivica u pozadini.
- **Povećanje sigma (npr. `sigma=3.0`)**
  - Slike su gladje, ivice su kontinualne, ali tanje ivice mogu nestati.
  - Manje lažnih ivica, ali manja preciznost kod finih detalja.
- **Smanjenje apsolutnih pragova**
  - Više ivica je detektovano -> uključuje šum i neželjene fragmente.
- **Povećanje apsolutnih pragova**
  - Samo najsnažnije ivice su sačuvane -> gubi se deo pravih detalja, ali manje lažnih ivica.

Izabrani parametri predstavljaju **balans između detekcije pravih ivica i eliminacije šuma**.

Prikazane ivice za slike Lena, Van i Camerman su **jasne i kontinuirane**, bez prevelikog broja fragmenata, što ih čini optimalnim za dalju obradu i analizu.

In [37]:

```
lena_edges = canny_edge_detection(img1, sigma = 2.0, abs_lower = 0.09, abs_upper = 0.23)
van_edges = canny_edge_detection(img2, sigma = 2.0, abs_lower = 0.09, abs_upper = 0.23)
camerman_edges = canny_edge_detection(img3, sigma = 2.0, abs_lower = 0.09, abs_upper = 0.23)

fig, axes = plt.subplots(nrows = 1, ncols = 3, dpi = 80, figsize = (11, 7))

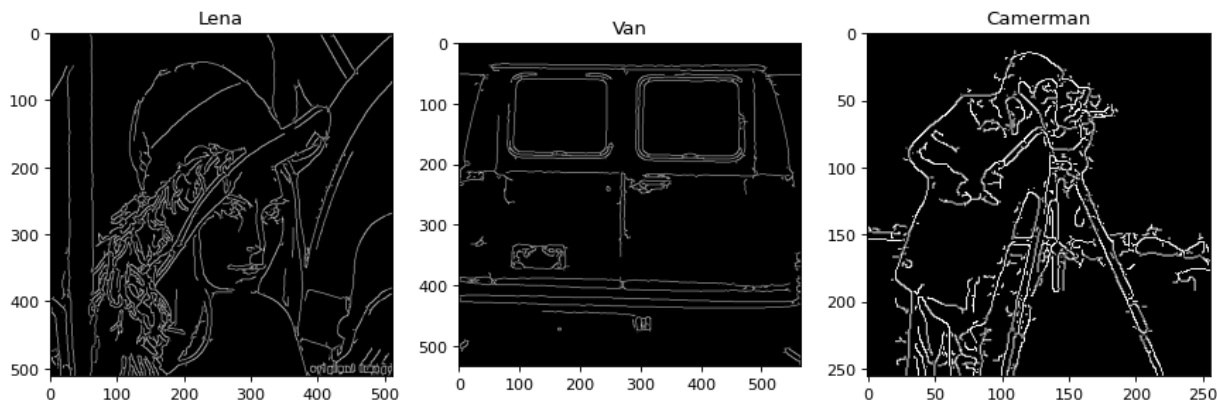
axes[0].imshow(lena_edges, cmap = 'gray')
axes[0].set_title("Lena")

axes[1].imshow(van_edges, cmap = 'gray')
axes[1].set_title("Van")

axes[2].imshow(camerman_edges, cmap = 'gray')
axes[2].set_title("Camerman")

plt.tight_layout()
plt.show()
```





## Zaključci

- Gaussian filter efikasno uklanja šum i priprema sliku.
- Gradijenti i magnituda identifikuju prelaze intenziteta.
- Kvantizacija uglova omogućava preciznu selekciju ivica.
- Histerezis thresholding kombinuje jake i slabe ivice, uklanjajući šum.
- Konačne ivice su tanke, jasno definisane i obuhvataju ključne konture slike Lenine.

Kod Canny detekcije ivica, glavni konture i detalji su uglavnom uspešno detektovani kod svih test primera.

- **Lena:** ivice lica i kose su jasno vidljive, slab šum u pozadini nije problem, ali postoji dodatni stub.
- **Van:** osnovne ivice automobila su detektovane, ali sitni detalji i refleksije su izgubljeni.
- **Cameraman:** konture figure i stativa su uhvaćene, ali manji detalji ruku i pozadine nisu potpuno vidljivi a postoji i jasno vidljiv šum na nekim delovima.

Neuspeh u detekciji sitnijih detalja verovatno je posledica niskog kontrasta, podešavanja pragova ( `abs_lower` i `abs_upper` ) ili same vrednosti sigma ( `sigma = 2.0` ).

Za precizniju detekciju potrebno je eksperimentisanje sa pragovima i lokalno podešavanje filtra za slabije ivice.

```
In [66]: def canny_usage(image, sigma=2.0, lt=0.09, ht=0.2):
            image = ski.util.img_as_float(image)
            result = ski.feature.canny(image, sigma=sigma, low_threshold=lt, high_threshold=ht)

            figure, ax = plt.subplots(nrows=1, ncols=2, figsize=(15, 10), dpi=80)
            ax[0].imshow(image, cmap='gray', vmin=0, vmax=1), ax[0].axis('off'), ax[0].set_title('Original Image')
            ax[1].imshow(result, cmap='gray'), ax[1].axis('off'), ax[1].set_title('Izdvajanje ivica')
            plt.tight_layout()
            plt.show()
            return
```

In [67]:

```
lena = ski.io.imread("../sekvence/lena.tif")  
van = ski.io.imread("../sekvence/van.tif")  
camerman = ski.io.imread("../sekvence/camerman.tif")  
  
canny_usage(lena)  
canny_usage(van)  
canny_usage(camerman)
```

Originalna slika



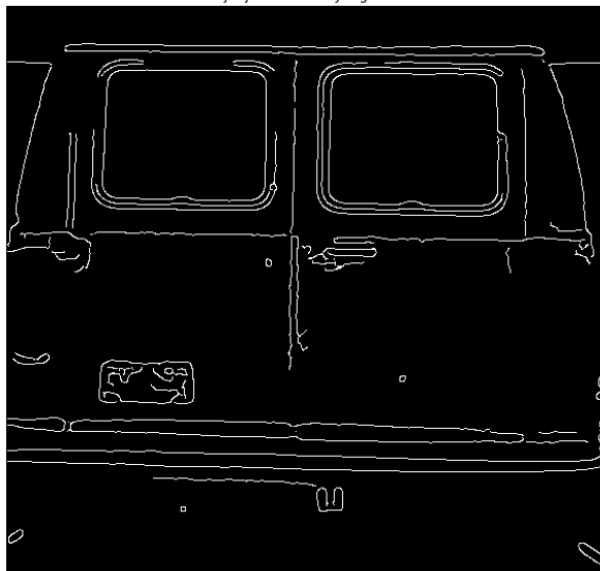
Izdvajanje ivica Canny algoritmom



Originalna slika



Izdvajanje ivica Canny algoritmom





## Uticaj parametara Canny detekcije na rezultate

### 1. Sigma ( $\sigma$ ) – širina Gauss filtera

- **Što je  $\sigma$  veće:**
  - Slika postaje **gladja**, šum je jače uklonjen.
  - Male ivice i sitni detalji **nestaju**.
  - Glavne konture ostaju, ali su **blago zamućene**.
- **Što je  $\sigma$  manje:**
  - Očuvaju se **sitne ivice** i detalji.
  - Šum postaje vidljiv, može se detektovati kao lažne ivice.

### 2. Upper threshold

- Viši prag → **samo najjače ivice** se uzimaju odmah.
- Smanjuje lažne ivice, ali može **izostaviti sitne, ali bitne konture**.

### 3. Lower threshold

- Niži prag → uključuje **slabe ivice** povezane sa jakim ivicama.
- Ako je previše nizak → **šum se uključuje**, loše za finalnu masku.
- Ako je previše visok → slabe ivice nisu zadržane → konture postaju **isprekidane**.
- Odnosi se na histerezis: slabe ivice moraju imati kontakt sa jakim da opstanu.

### 4. Kombinacija parametara

- **Veliko  $\sigma$  + visok upper + visok lower** → samo glavne ivice, glatka ali "rigidna" slika ivica.
- **Malo  $\sigma$  + nizak lower** → mnogo detalja, sitni šum vidljiv, tanke ivice prate sve prelaze.

- **Optimalno:** srednje  $\sigma$ , upper prag za glavne ivice = 0.2 otprilike, lower prag  $\sim 0.4-0.5 \times$  upper da slabe ivice budu sačuvane.

## 5. Vizuelni efekti

- Povećanje  $\sigma \rightarrow$  ivice "deblje", manje detalja.
- Povećanje upper prag  $\rightarrow$  gube se jake ivice.
- Povećanje lower prag  $\rightarrow$  slabe ivice nestaju, konture postaju diskontinuirane.

## 3) DEO(BONUS)

In [100...

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt
from scipy import ndimage as ndi
from skimage.morphology import closing, disk
import scipy as sp
```

Funkcija `extractLargestRegion` služi za izdvajanje **najvećeg povezanog regiona** iz binarne maske slike. Ova operacija eliminiše male, neželjene regione (šum) i zadržava samo dominantni objekat.

## Princip rada funkcije

1. **Labeliranje regiona:** svakom povezanom regionu se dodeljuje jedinstvena oznaka, pozadina se ignoriše.
2. **Ekstrakcija svojstava regiona:** izračunava se površina i ostala svojstva svakog regiona.
3. **Provera postojanja regiona:** ako nema nijednog regiona, vraća se originalna maska.
4. **Pronalaženje najvećeg regiona:** bira se region sa najvećom površinom.
5. **Formiranje izlazne maske:** kreira se nova binarna maska u kojoj su aktivni samo pikseli najvećeg regiona.

Ova funkcija se koristi kada želimo izrezati jedino drvo iz cele slike.

In [81]:

```
def extractLargestRegion(treeMask):  
    # Labeliranje povezanih regiona  
    labeled_image = ski.measure.label(treeMask, background=0)  
    regions = ski.measure.regionprops(labeled_image)  
  
    if not regions:  
        print("Nema detektovanih regiona!")  
        return treeMask # vrati originalnu masku ako ništa nije detektovano  
  
    # Pronađi region sa najvećom površinom  
    largest_region = regions[0]  
    for region in regions:  
        if region.area > largest_region.area:  
            largest_region = region  
  
    # Napravi masku samo za najveći region  
    largestMask = (labeled_image == largest_region.label)  
  
    return largestMask
```

Funkcija `extractTreeOnly` ekstrahuje glavni objekat stabla iz grayscale ili bilateralne slike. Prvo koristi **Canny detekciju ivica** (  $\sigma=1.0$  , pragovi 0.1–0.2) za pronalaženje kontura stabla. Zatim se primenjuju **morfološke operacije**: `binary_dilation` spaja fragmentisane ivice, `binary_erosion` uklanja šum, a `closing` i `binary_fill_holes` popunjavaju rupe i prekide. Rezultat se pretvara u binarnu masku koja se prosleđuje funkciji `extractLargestRegion` kako bi ostao samo najveći povezani region – glavno stablo. Na kraju, maska se primenjuje na originalnu sliku, zadržavajući samo piksele stabla.

Broj iteracija u morfološkim operacijama direktno utiče na kvalitet: manji broj iteracija može ostaviti praznine i fragmentisane ivice, dok veći broj spaja sve fragmente, ali povećava vreme izvršavanja i može uključiti neželjene objekte. Optimalan balans između preciznosti i performansi bira se eksperimentalno, zavisno od veličine i kvaliteta slike.

In [82]:

```
def extractTreeOnly(bilateralTree):
    # Canny detekcija ivica
    edges = ski.feature.canny(bilateralTree, sigma=1.0, low_threshold=0.1, high_thres

    # Morfološki koraci da se spoje fragmenti i popune rupe
    dilated = ndi.binary_dilation(edges, iterations=3)
    eroded = ndi.binary_erosion(dilated, iterations=2)
    closedMask = closing(eroded, disk(5))
    filledMask = ndi.binary_fill_holes(closedMask)

    # Napravi binarnu masku za labeliranje
    binaryMask = filledMask > 0

    # Izdvajanje najvećeg regiona
    largestMask = extractLargestRegion(binaryMask)

    # Primena maske na originalni grayscale/bilateral image
    treeOnly = bilateralTree * largestMask

    return treeOnly
```

## Pith Localization: Hessian-Based Ridge Detection i Gradient Analysis

Ovaj deo metodologije se koristi za **lokalizaciju centra stabla na slici preseka stabla**. Ključni izazov je da prstenovi stabla nisu savršeno kružni i mogu biti deformisani, a pith možda nije u centru zbog neujednačenog rasta stabla.

Proces se sastoji od tri glavna koraka: **Hessian-based ridge detection, gradient analysis i weighted radial voting**.

### 1. Hessian-Based Ridge Detection

Funkcija `hessianBasedRidgeDetection(I, beta, c)` detektuje **ridge-ove**(prstenove stabla).

Koraci funkcije:

1. Filtar se primenjuje na više vrednosti `sigma` od 1 do 3 kako bi se detektovali prstenovi različitih debljina.
2. Za svaki piksel se računa Hessian matrica:

$$H(x, y) = \begin{bmatrix} \frac{\partial^2 I}{\partial x^2} & \frac{\partial^2 I}{\partial x \partial y} \\ \frac{\partial^2 I}{\partial y \partial x} & \frac{\partial^2 I}{\partial y^2} \end{bmatrix}$$

3. Izračunavamo sopstvene vrednosti (eigenvalues) gde `lambda1` i `lambda2` označavaju

zakrivljenost u različitim pravcima. Sortiramo ih tako da je

$$|\lambda_1| \leq |\lambda_2|.$$

4. Na osnovu eigenvalues se računa  $RB$  (blobness ratio) i  $S$  (structureness). Konačna vrednost ridge map-e  $V$  se računa na sledeći način:

- Ako je  $\lambda_2 > 0 \rightarrow V = 0$  (piksel nije ridge)
- Ako je  $\lambda_2 \leq 0$ :

$$V = \exp\left(-\frac{RB^2}{2\beta^2}\right) \cdot \left(1 - \exp\left(-\frac{S^2}{2c^2}\right)\right)$$

1. Rezultat za svaku skalu se kombinuje pomoću `np.maximum.reduce` da bi se dobila **finalna ridge mapa** sa naglašenim prstenovima stabla.

---

## 2. Gradient Analysis

Funkcija `ScharrGradient(I)` računa **magnitude i pravac gradijenta** prstenova stabla.

Koraci:

1. Definišemo Scharr kernel:  $K_x$  i  $K_y$  za precizno određivanje pravca normalnog na prsten.
  2. Uradimo Konvoluciju:  $G_x = I \otimes K_x$  (horizontalni gradijent),  $G_y = I \otimes K_y$  (vertikalni gradijent).
  3. Izračunamo Magnitudu i pravac:  $G = \sqrt{G_x^2 + G_y^2}$ ,  $\theta = \arctan2(G_y, G_x)$ .
- 

## 3. Veza sa pith lokalizacijom

Kombinovanjem Hessian ridge detectiona i Scharr gradijenta:

- Ridge mapa naglašava prstenove stabla.
- Gradijent daje smer prema unutrašnjem centru prstena.
- Sledeći korak (weighted radial voting) koristi ove informacije da pronađe **tačnu poziciju pith-a**, po algoritmu koji radi egzaktno čak i na deformisanim stablima.



In [83]:

```

def hessianBasedRidgeDetection(I, beta, c):
    """
    Docstring for hessianBasedRidgeDetection

    :param I: Description
    :param beta: Description
    :param c: Description
    """
    sigmas = np.arange(1, 4)
    regMaps = []

    for sigma in sigmas:
        # drugi izvod [[Ixx, Ixy], [Ixy, Iyy]]
        # order = po kojoj dimenziji se radi izvod i koliko puta
        # primer: Ixx = 2 puta izvod po x, 0 po y
        Ixx, Ixy, Iyy = ski.feature.hessian_matrix(I, sigma=sigma, order='rc', use_grad=True)

        # Lambda, RB, S
        lambda1, lambda2 = ski.feature.hessian_matrix_eigvals((Ixx, Ixy, Iyy))
        l1 = lambda1.copy()
        l2 = lambda2.copy()

        # one vrednosti piksela koje su pogresne zamenimo
        mask = np.abs(l1) > np.abs(l2)
        l1[mask], l2[mask] = l2[mask], l1[mask]

        epsilon = 1e-8
        RB = np.abs(l1) / (np.abs(l2) + epsilon)
        S = np.sqrt(np.square(l1) + np.square(l2))

        # V
        VMask = l2 > 0

        V = np.zeros_like(S)
        V[~VMask], V[VMask] = np.exp(-(RB[~VMask] ** 2 / 2 * beta)) * (1 - np.exp(-S[VMask]))

        regMaps.append(V)

    # spajanje svih mapa u jednu finalnu ridge mapu
    ridgeMultiScale = np.maximum.reduce(regMaps)

    return ridgeMultiScale

```

In [84]:

```
def ScharrGradient(I):
    Kx = np.array([[ -3,  0,  3],
                   [-10, 0, 10],
                   [ -3,  0,  3]])

    Ky = np.array([[ -3, -10, -3],
                   [  0,  0,  0],
                   [  3, 10,  3]])

    Gx = ndi.correlate(I, Kx, mode="nearest")
    Gy = ndi.correlate(I, Ky, mode="nearest")
    G = np.sqrt(np.square(Gx) + np.square(Gy))
    theta = np.arctan2(Gy, Gx)

    return Gx, Gy, G, theta
```

## Pith Localization: Weighted Radial Voting

Nakon Hessian ridge detekcije i gradijent analize, lokalizacija **pith-a** koristi **Weighted Radial Voting**. Metoda kombinuje ridge mapu i smer gradijenta da pronađe centar stabla.

### Princip rada

1. Kreira se **akumulator** iste dimenzije kao slika.
2. **Glasački pikseli**: svi pikseli gde `ridgeMap > 0.05`.
3. **Weighted voting**: svaki piksel "glasava" duž pravca gradijenta, koristeći težinu `ridgeMap`. Kandidati van slike se ignorišu.
4. **Gaussian filter** ( `sigma=2` ) glatko spaja glasove i naglašava centar jer se može desiti da više centara bude izglasano.
5. **Pith** je maksimum u filtriranom akumulatoru.

In [85]:

```

def weightedRadialVoting(origImg, ridgeMap, theta):
    """
    Brža, vektorizovana verzija Weighted Radial Voting-a.
    origImg : slika (samo dimenzije potrebne)
    ridgeMap : ridge mapa dobijena Hessian detekcijom
    theta    : matrica gradijenata (ugao) za svaki piksel
    """

    accumulator = np.zeros_like(ridgeMap)
    voterPoints = np.argwhere(ridgeMap > 0.05) # pikseli koji glasaju

    max_dist = max(origImg.shape)
    t_vals = np.arange(0, max_dist) # niz distanci

    for yi, xi in voterPoints:
        gradAngle = theta[yi, xi]
        weight = ridgeMap[yi, xi]

        # vektorizovano računanje candidate koordinata
        x_candidates = np.round(xi + t_vals * np.cos(gradAngle)).astype(int)
        y_candidates = np.round(yi + t_vals * np.sin(gradAngle)).astype(int)

        # maska za ostajanje unutar slike
        mask = (x_candidates >= 0) & (x_candidates < origImg.shape[1]) & \
            (y_candidates >= 0) & (y_candidates < origImg.shape[0])

        x_candidates = x_candidates[mask]
        y_candidates = y_candidates[mask]

        # vektorizovano sabiranje težina u akumulator
        np.add.at(accumulator, (y_candidates, x_candidates), weight)

    # Gaussian filter za smanjenje više piksela i isticanje centra
    gaussianAcc = ndi.gaussian_filter(accumulator, sigma=2)

    # pronalaženje pitha
    pithY, pithX = np.unravel_index(np.argmax(gaussianAcc), accumulator.shape)
    return pithX, pithY

```

Funkcija `imgRoi` se koristi za izdvajanje uzoraka stabla iz slike preseka, slično korišćenju **increment borera** u praksi.

Umesto da se broje prstenovi preko cele površine, slika se seče u četiri uske trake koje počinju od detektovanog pith-a i protežu se do kore stabla.

Trake se izdvajaju u četiri pravca: **north, east, south i west**, kako bi se obuhvatio kompletan uzorak prstenova.

Veličina ROI-a određena je procentom dimenzija slike ( `percentVal` ), što omogućava balans između preciznosti i vremena obrade.

Izdvojene trake se zatim horizontalno poravnavaju i tretiraju kao linearni vremenski niz od pith-a ka kori.

Ove ROI slike se koriste za automatsko brojanje prstenova u narednom koraku analize. Metoda omogućava fokus na relevantne delove slike i smanjuje šum sa periferije.

Početna vrednost radijalnog sečenja je 2% širine slike, što je dovoljno za tačno detektovanje prstenova.

Ako je ROI previše mali, vrednost procenta se može povećati za bolju pokrivenost.

Cilj je simulirati uzorak stabla i pripremiti ga za preciznu analizu prstenova.

In [86]:

```
def imgRoi(I, pithX, pithY, percentVal):
    Ny, Nx = I.shape

    # horizontalna traka centra -> istok
    rowStart = max(0, pithY - (percentVal * Ny) // 100)
    rowEnd = min(Ny, pithY + (percentVal * Ny) // 100)
    eastRoi = I[rowStart:rowEnd, pithX:]

    # horizontalna traka centar-> zapad
    rowStart = max(0, pithY - (percentVal * Ny) // 100)
    rowEnd = min(Ny, pithY + (percentVal * Ny) // 100)
    westRoi = I[rowStart:rowEnd, : pithX]

    # vertikalna traka centar-> sever
    colStart = max(0, pithX - (percentVal * Nx) // 100)
    colEnd = min(Nx, pithX + (percentVal * Nx) // 100)
    northRoi = I[:pithY, colStart:colEnd]

    # vertikalna traka centar -> jug
    colStart = max(0, pithX - (percentVal * Nx) // 100)
    colEnd = min(Nx, pithX + (percentVal * Nx) // 100)
    southRoi = I[pithY:, colStart:colEnd]

    return eastRoi, westRoi, northRoi, southRoi
```

Funkcija RingDetection automatski broji **prstenove stabla** u izabranom ROI-u.

## Princip rada:

### 1. Intensity profile:

Računa se suma intenziteta piksela po horizontali ili vertikali, zavisno od dimenzija ROI-a, da bi se dobio 1D profil slike.

### 2. Smoothing:

Profil se glatko filtrira Gausovim filterom kako bi se uklonio šum i manje varijacije.

### 3. Normalizacija i inverzija:

Profil se normalizuje na [0,1] i invertuje, tako da udubljenja (valleys) postaju vrhovi (peaks).

### 4. Detekcija vrhova:

Korišćenjem `scipy.signal.find_peaks` funkcije se detektuju vrhovi sa zadatim pragom visine ( `heightLevel` ) i odgovarajućom visinom.

### 5. Brojanje prstenova:

Broj detektovanih vrhova odgovara broju prstenova u ROI-u.

In [87]:

```
def RingDetection(roi, sigma, heightLevel):
    H, W = roi.shape

    # 1) intensity profile
    if H < W:
        profile = np.sum(roi, axis=0)
    else:
        profile = np.sum(roi, axis=1)

    # 2) smoothing
    smoothedProfile = ndi.gaussian_filter1d(profile, sigma=sigma)

    # 3) normalize to [0,1] and invert (valleys -> peaks)
    smoothedProfile = (smoothedProfile - smoothedProfile.min()) / (smoothedProfile.max() - smoothedProfile.min())
    invProfile = 1 - smoothedProfile

    # 4) find peaks using scipy
    peaks, properties = sp.signal.find_peaks(invProfile, height=heightLevel, prominence=heightLevel)

    # broj vrhova
    return len(peaks)
```

## Funkcija RingDetection rezervna

**Napomena:** Ovo su **rezervne funkcija** sa **ručnom implementacijom pronalaženja vrhova**, gde se koristi samo `heightLevel` radi jednostavnosti za detekciju. Ostali kriterijumi poput `prominence`, i minimalne udaljenosti primenjeni su u rezervnoj funkciji 2.

In [17]:

```
# def RingDetection(roi, sigma, heightLevel=0.05, minProminence = 0.000001):
#     """
#     Jednostavna detekcija prstenova.
#     Samo broji lokalne vrhove koji prelaze minimalni heightLevel.
#     """
#
#     H, W = roi.shape
#
#     # 1) intensity profile
#     profile = np.sum(roi, axis=0) if H < W else np.sum(roi, axis=1)
#
#     # 2) smoothing
#     smoothedProfile = ndi.gaussian_filter1d(profile, sigma=sigma)
#
#     # 3) normalizacija i invertovanje (doline -> vrhovi)
#     smoothedProfile = (smoothedProfile - smoothedProfile.min()) / (smoothedProfile.max() - smoothedProfile.min())
#     invProfile = 1 - smoothedProfile
#
#     # 4) pronalaženje vrhova po visini
#     peaks = 0
#     for i in range(1, len(invProfile) - 1):
#         if invProfile[i] >= invProfile[i - 1] and invProfile[i] >= invProfile[i + 1]:
#             if invProfile[i] >= heightLevel:
#                 peaks += 1
#
#     return peaks
```

In [ ]:

```

# def RingDetection(roi, sigma, heightLevel, minProminence):
#     H, W = roi.shape

#     # 1) intensity profile
#     if H < W:
#         profile = np.sum(roi, axis=0)
#     else:
#         profile = np.sum(roi, axis=1)

#     # 2) smoothing
#     smoothedProfile = ndi.gaussian_filter1d(profile, sigma=sigma)

#     # 3) invert (valleys -> peaks)
#     smoothedProfile = np.clip(smoothedProfile, 0, 1)
#     invProfile = 1 - smoothedProfile

#     # MINIMUM DISTANCE (iz rada)
#     minDistance = int(np.std(invProfile))
#     if minDistance < 1:
#         minDistance = 1

#     peak_indices = []
#     peak_heights = []

#     # 4) peak kandidati (height + prominence)
#     for i in range(1, len(invProfile) - 1):
#         if invProfile[i] >= invProfile[i-1] and invProfile[i] >= invProfile[i+1]:

#             height = invProfile[i]
#             if height < heightLevel:
#                 continue

#             prominence = height - max(invProfile[i-1], invProfile[i+1])
#             if prominence < minProminence:
#                 continue

#             peak_indices.append(i)
#             peak_heights.append(height)

#     # 5) MIN DISTANCE FILTER (samo susedni)
#     final_peaks = []
#     for idx, h in zip(peak_indices, peak_heights):
#         if len(final_peaks) == 0:
#             final_peaks.append((idx, h))
#         else:
#             prev_idx, prev_h = final_peaks[-1]
#             if idx - prev_idx < minDistance:
#                 # zadrži viši pik
#                 if h > prev_h:
#                     final_peaks[-1] = (idx, h)
#             else:
#                 final_peaks.append((idx, h))

#     return len(final_peaks)

```



# Funkcija ringCountConsolidation

Funkcija kombinuje broj prstenova dobijen sa četiri ROI trake (north, south, east, west) u **jedan finalni broj**.

Princip rada:

1. Računa se **medijana** i **standardna devijacija** od četiri vrednosti.
2. Zadržavaju se samo vrednosti koje zadovoljavaju uslov:

$$|x_i - \text{medijana}| < 1.5 \cdot SD$$

čime se uklanjaju očigledne loše vrednosti.

3. Finalni broj prstenova je **prosek validnih vrednosti**, zaokružen na najbliži ceo broj.

```
In [88]: def ringCountConsolidation(rings):
  """
  Docstring for ringCountConsolidation

  :param rings: Lista broja krugova kao ulazni argument za sve 4 strane sever, jug,
  """

  # 1. izracunamo medijanu sve 4 vrednosti
  ringsMedian = np.median(rings)

  # 2. izracunamo standardnu devijaciju sve 4 vrednosti
  ringsStd = np.std(rings)

  # 3. stavimo u listu validnih brojeve samo one koji zadovoljavaju sledecu formulu

  validRings = []
  for ring in rings:
      if np.abs(ring - ringsMedian) < 1.5 * ringsStd:
          validRings.append(ring)

  finalNumOfRings = np.floor(np.mean(validRings) + 0.5)

  return finalNumOfRings
```

## Evaluacija pith lokalizacije i brojanja prstenova

Ove funkcije služe za **kvantitativnu procenu tačnosti detekcije pith-a i broja prstenova** na datasetu, u skladu sa poglavljem 2.7.

## Funkcija pithEvaluation

Procena tačnosti detekcije pith-a u odnosu na **ground truth (GTPith)**.

- **MDE (Mean Distance Error)**: prosečna udaljenost detektovanog pith-a od stvarnog centra.

$$MDE = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i - x_{idet})^2 + (y_i - y_{idet})^2}$$

- **STD**: standardna devijacija udaljenosti, meri varijabilnost greške.
- **RMSE (Root Mean Squared Error)**: kvadratni koren prosečne kvadratne greške, naglašava veće odstupanja.
- **Hit rate**: procent pith-a koji se detektuju unutar tolerancije

$$(T = \alpha \cdot CWidth).$$

## Funkcija TreeRingCountingEvaluation

Evaluacija predviđenog broja prstenova po slici.

- **ME (Mean Error)**: prosečna razlika predviđenog i stvarnog broja prstenova.
- **MAE (Mean Absolute Error)**: prosečna apsolutna razlika, meri preciznost predviđanja.
- **RMSE**: koren prosečne kvadratne greške između stvarnog i predviđenog broja.
- **MAPE (Mean Absolute Percentage Error)**: prosečna greška u procentima.
- **R<sup>2</sup> (Coefficient of Determination)**: meri koliko predviđeni broj prstenova objašnjava varijansu stvarnog broja.

## Funkcija restOfTheParameters

Računa dodatne metrike za evaluaciju broja prstenova:

- **Precision**: koliko predviđenih prstenova je tačno detektovano.

$$Precision = \frac{\sum TP}{\sum (TP + FP)}$$

- **Recall**: koliko stvarnih prstenova je detektovano.

$$Recall = \frac{\sum TP}{\sum (TP + FN)}$$

- **F1 Score**: harmonijska sredina Precision i Recall-a.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **Image-level Accuracy:** procenat slika sa tačno predviđenim brojem prstenova.
- **Ring-level Accuracy:** procenat pravilno detektovanih prstenova od ukupnog broja prstenova u datasetu.

In [89]:

```
def pithEvaluation(GTPith, detectedPith, testImages, alpha):
    diList = []
    for i in range(len(testImages)):
        # 1. distance error
        xi, yi = GTPith[i]
        xiDetected, yiDetected = detectedPith[i]
        dx = xiDetected - xi
        dy = yiDetected - yi
        di = np.sqrt(dx ** 2 + dy ** 2)
        diList.append(di)

    # 2. mean distance error
    MDE = np.mean(diList)

    # 3. standardna devijacija slika
    STD = np.std(diList)

    # 4. RMSE
    RMSE = np.sqrt(1 / len(testImages) * np.sum([di ** 2 for di in diList]))

    # 5 HRate:
    HWList = []
    for img in testImages:
        H, W = img.shape[:2]
        HWList.append(min(H, W))

    # threshold i CWidth
    CWidth = 0.02 * min(HWList)
    T = np.floor(alpha * CWidth) - 1

    # racunanje hitRate
    hits = 0
    for di in diList:
        if di <= T:
            hits += 1

    hitRate = (hits / len(testImages)) * 100

    return {
        "MDE": MDE,
        "STD": STD,
        "RMSE": RMSE,
        "Hit rate": hitRate
    }
```

In [93]:

```
def TreeRingCountingEvaluation(rings, testImages):
    """
    Funkcija za evaluaciju broja prstenova po datasetu (prvih 5 metrika).

    :param rings: lista tuple-ova [(numOfRings, estimatedNumOfRings), ...] po slici
    :return: dictionary sa ME, MAE, RMSE, MAPE, R2
    """

    # izdvajanje stvarnog i predviđenog broja prstenova
    numRingsList = np.array([num for num, est in rings])
    estRingsList = np.array([est for num, est in rings])
    N = len(rings)

    # 1 ME
    ME = np.mean(estRingsList - numRingsList)

    # 2 MAE
    MAE = np.mean(np.abs(estRingsList - numRingsList))

    # 3 RMSE
    RMSE = np.sqrt(np.mean((estRingsList - numRingsList) ** 2))

    # 4 MAPE
    MAPE = 100 * np.mean(np.abs(estRingsList - numRingsList) / numRingsList)

    # 5 R²
    den = np.sum((numRingsList - np.mean(numRingsList)) ** 2)
    RSquared = np.nan if den == 0 else 1 - np.sum((numRingsList - estRingsList) ** 2) / den

    firstFive = {"ME": ME, "MAE": MAE, "RMSE": RMSE, "MAPE": MAPE, "R2": RSquared}
    secondFive = restOfTheParameters(numRingsList, estRingsList, testImages)
    return firstFive, secondFive
```

In [92]:

```
def restOfTheParameters(numRingsList, estRingsList, testImages):
    """
    Računa Precision, Recall, F1, Accuracy_image i Accuracy_ring
    :param numRingsList: np.array stvarni brojevi prstenova po slici
    :param estRingsList: np.array predviđeni brojevi prstenova po slici
    :param testImages: lista test slika
    :return: dict sa metrima
    """
    N = len(testImages)

    numRingsList = np.array(numRingsList)
    estRingsList = np.array(estRingsList)

    # 1. TP, FP, FN element-wise - dobijem listu TPi, FPi, FNi itd
    TP = np.minimum(estRingsList, numRingsList) # koliko prstenova je pravilno
    FP = np.maximum(estRingsList - numRingsList, 0) # lažno dodati prstenovi
    FN = np.maximum(numRingsList - estRingsList, 0) # propušteni prstenovi

    # 2. Precision, Recall, F1
    Precision = np.sum(TP) / (np.sum(TP + FP)) if np.sum(TP + FP) > 0 else 0
    Recall = np.sum(TP) / (np.sum(TP + FN)) if np.sum(TP + FN) > 0 else 0
    F1 = 2 * Precision * Recall / (Precision + Recall) if (Precision + Recall) > 0 else 0

    # 3. Image-Level Accuracy
    accuracyImg = np.sum(estRingsList == numRingsList) / N

    # 4. Ring-Level Accuracy
    accuracyRing = np.sum(TP) / np.sum(numRingsList)

    return {
        "Precision": Precision,
        "Recall": Recall,
        "F1": F1,
        "Accuracy_image": accuracyImg,
        "Accuracy_ring": accuracyRing
    }
```

Funkcija `visualize_tree` Prikazuje sliku stabla sa označenim pith-om i finalnim brojem prstenova.

In [94]:

```
def visualize_tree(treeOnly, pithX, pithY, finalRings, ax=None):
    """
    Crta jednu sliku sa pith-om i finalnim brojem prstenova.
    Može da se koristi sa subplot-om (ako je ax != None).
    """
    if ax is None:
        fig, ax = plt.subplots(figsize=(9, 6), dpi=80)

    ax.imshow(treeOnly, cmap="gray")
    ax.scatter([pithX], [pithY], color='red', s=50, label='Pith (centar)')
    ax.legend()
    ax.axis('off')
    ax.set_title(f"Finalni broj prstenova: {finalRings}")
    plt.show()
```

# Glavna logika algoritma

Ova funkcija implementira **ceo pipeline za automatsko brojanje prstenova** sa slike stabla:

## 1. Priprema slike:

- Pretvara RGB sliku u grayscale
- Uklanja šum bilateralnim filterom

## 2. Ekstrakcija stabla:

- Detektuje i izdvoji samo deo slike koji pripada stablu

## 3. Pith lokalizacija:

- Hessian-based ridge detekcija naglašava linije prstenova
- Scharr operator određuje gradient i pravac linija
- Weighted Radial Voting određuje centar (pith) stabla

## 4. ROI ekstrakcija:

- Iz centra se seku četiri trake (north, east, south, west) koje sadrže prstenove

## 5. Detekcija prstenova po ROI:

- Konvertuje trake u 1D intenzitet profil
- Glatko filtrira i pronalazi prstenove

## 6. Kombinovanje broja prstenova:

- Kombinuje detekcije sa svih četiri strana
- Uklanja odstupajuće vrednosti i računa finalni broj prstenova

## Izlaz funkcije:

- `finalRings` – broj prstenova
- `pithX`, `pithY` – koordinata centra stabla
- `treeOnly` – binarna maska stabla za vizualizaciju

In [98]:

```

def extract_tree_age(treeImg, beta, c, sigma=2, heightLevel=0.1, roiPercent=30):
    if treeImg.ndim != 3:
        raise ValueError("Only RGB images are supported")
    if treeImg.dtype == 'uint8':
        treeImg = ski.util.img_as_float(treeImg)

    # 1. Grayscale
    grayscaleTree = ski.color.rgb2gray(treeImg)

    # 2. Bilateral filter
    bilateralTree = ski.restoration.denoise_bilateral(grayscaleTree, sigma_spatial=2.

    # 3. Izdvoj drvo
    treeOnly = extractTreeOnly(bilateralTree)

    # 4. Hessian ridge detection
    ridgeMap = hessianBasedRidgeDetection(treeOnly, beta, c)

    # 5. Gradient analiza
    _, _, _, theta = ScharrGradient(ridgeMap)

    # 6. Pronađi pith
    pithX, pithY = weightedRadialVoting(treeOnly, ridgeMap, theta)

    # 7. Izreži ROI
    eastRoi, westRoi, northRoi, southRoi = imgRoi(treeOnly, pithX, pithY, roiPercent)

    # 8. Detekcija prstenova po ROI
    rings_east = RingDetection(eastRoi, sigma, heightLevel)
    rings_west = RingDetection(westRoi, sigma, heightLevel)
    rings_north = RingDetection(northRoi, sigma, heightLevel)
    rings_south = RingDetection(southRoi, sigma, heightLevel)

    # 9. Konsolidacija prstenova
    finalRings = ringCountConsolidation([rings_north, rings_east, rings_south, rings_

    return finalRings, pithX, pithY, treeOnly

```

In [96]:

```

# Učitavanje
treeImgKenne12 = ski.io.imread("../sekvence/kennel_img2.jpg")
treeImgKenne15 = ski.io.imread("../sekvence/kennel_img5.jpg")
treeImgKenne17 = ski.io.imread("../sekvence/kennel_img7.jpg")

treeUrudendroF03a = ski.io.imread("../sekvence/F03a.png")
treeUrudendroL08d = ski.io.imread("../sekvence/L08d.png")
treeUrudendroF02b = ski.io.imread("../sekvence/F02b.png")

```

In [101...

```
# Primena
numRings2, pithX2, pithY2, treeOnly2 = extract_tree_age(treeImgKennel2, beta=0.5, c=
numRings5, pithX5, pithY5, treeOnly5 = extract_tree_age(treeImgKennel5, beta=0.5, c=
numRings7, pithX7, pithY7, treeOnly7 = extract_tree_age(treeImgKennel7, beta=0.5, c=

numRingsF03a, pithXF03a, pithYF03a, treeOnlyF03a = extract_tree_age(treeUrudendroF03a
numRingsL08d, pithXL08d, pithYL08d, treeOnlyL08d = extract_tree_age(treeUrudendroL08d
numRingsF02b, pithXF02b, pithYF02b, treeOnlyF02b = extract_tree_age(treeUrudendroF02b
```

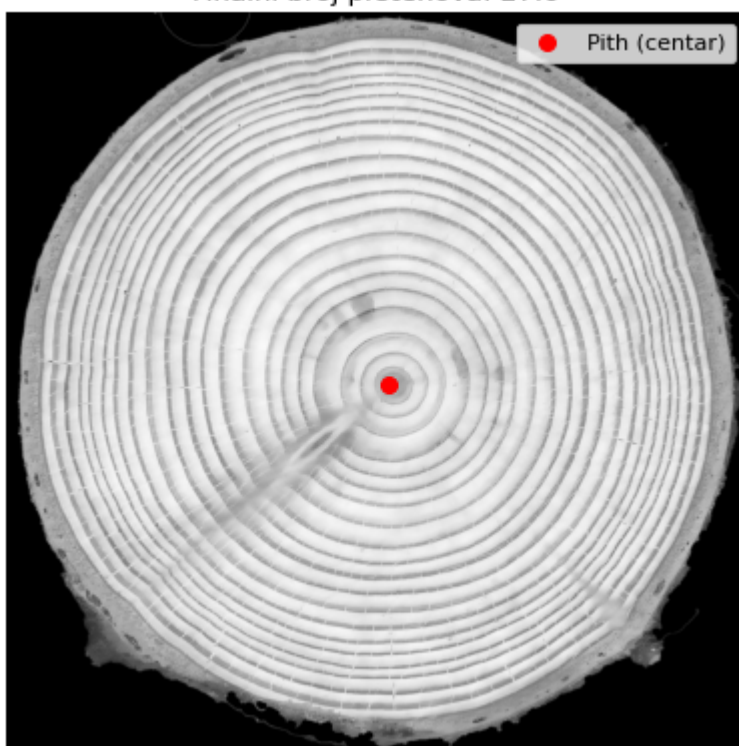
In [103...

```
# vizuelizacija drveca

visualize_tree(treeOnly2, pithX2, pithY2, numRings2)
visualize_tree(treeOnly5, pithX5, pithY5, numRings5)
visualize_tree(treeOnly7, pithX7, pithY7, numRings7)

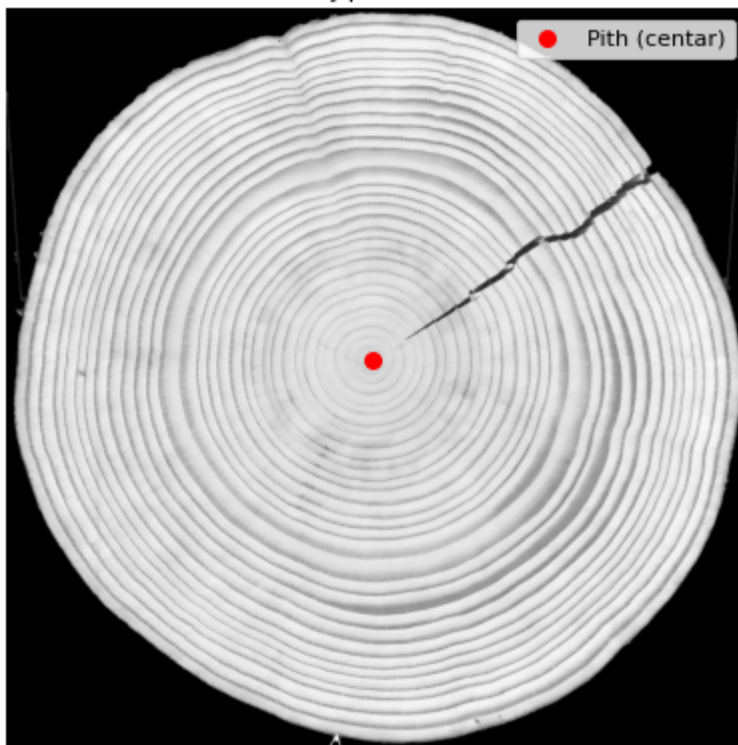
visualize_tree(treeOnlyF03a, pithXF03a, pithYF03a, numRingsF03a)
visualize_tree(treeOnlyL08d, pithXL08d, pithYL08d, numRingsL08d)
visualize_tree(treeOnlyF02b, pithXF02b, pithYF02b, numRingsF02b)
```

Finalni broj prstenova: 27.0

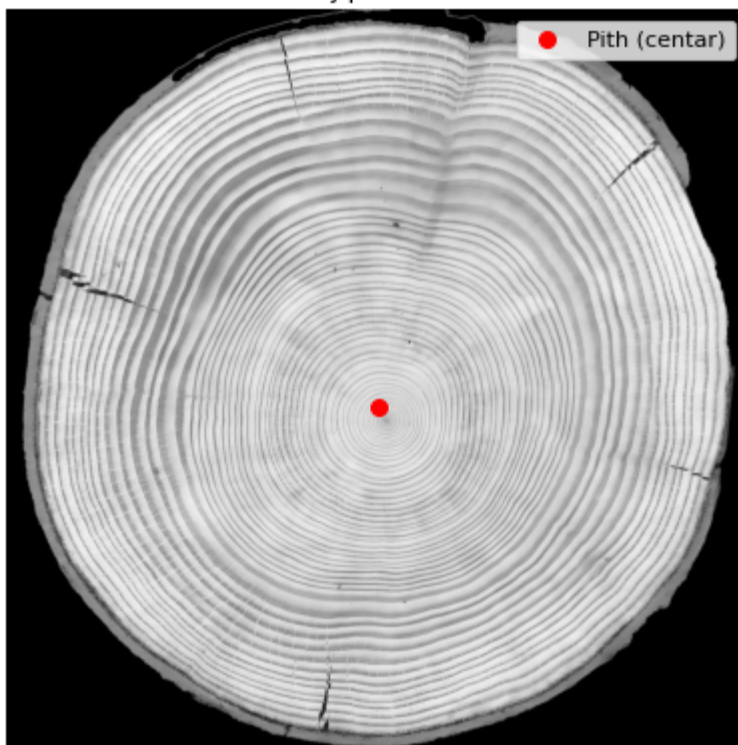




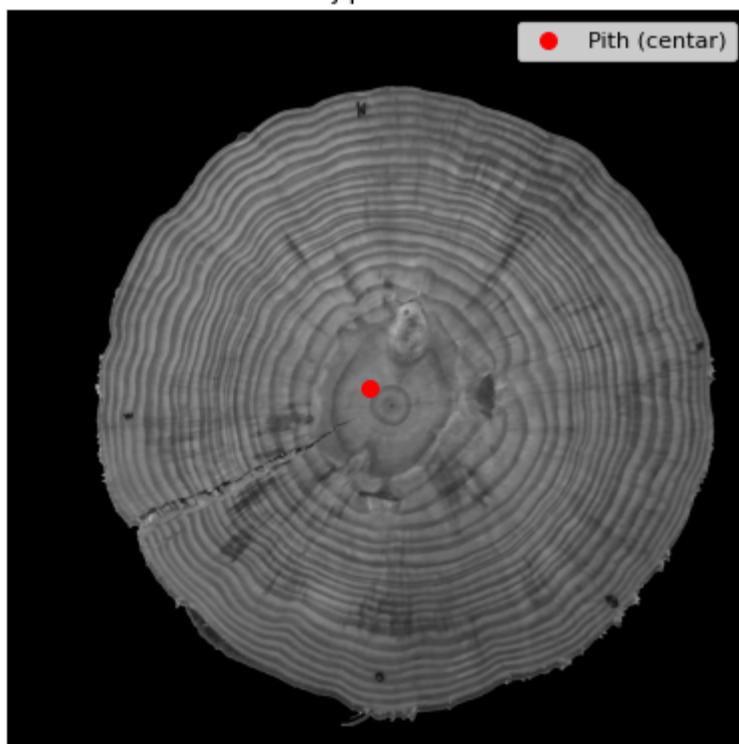
Finalni broj prstenova: 28.0



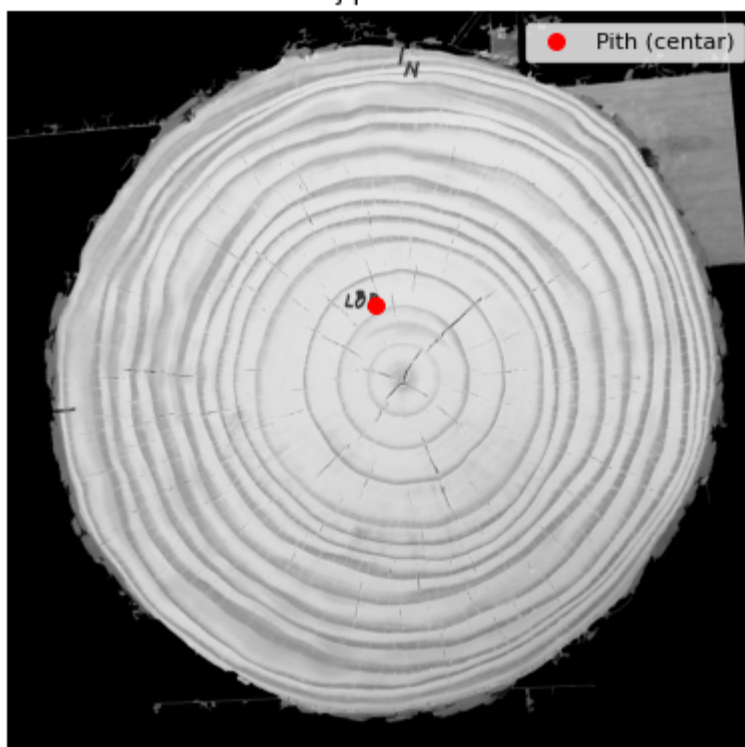
Finalni broj prstenova: 42.0



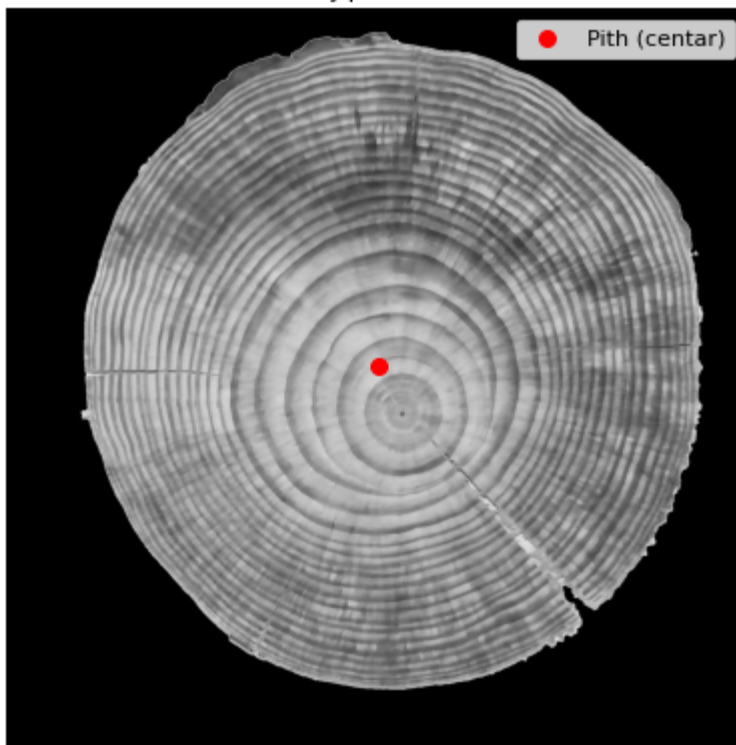
Finalni broj prstenova: 39.0



Finalni broj prstenova: 44.0



Finalni broj prstenova: 36.0



In [104...

```
# citanje parametara rucno sa slike i iz funkcije
realNumOfRings = np.array([22, 31, 49, 24, 16, 24])
estNumOfRings = np.array([numRings2, numRings5, numRings7, numRingsF03a, numRingsL08d, numRingsF02b])

realCoords = np.array([(664, 640), (626, 606), (665, 707), (1292, 1333), (1438, 1340), (1292, 1333)])
estCoords = np.array([(pithX2, pithY2), (pithX5, pithY5), (pithX7, pithY7),
                      (pithXF03a, pithYF03a), (pithXL08d, pithYL08d), (pithXF02b, pithYF02b)])

testImages = [
    treeImgKennel2, treeImgKennel5, treeImgKennel7,
    treeUrudendroF03a, treeUrudendroL08d, treeUrudendroF02b
]
```

In [105...

```
# poziv funkcije i statistika uspesnosti:
# 1 Evaluacija pitha
alpha = 0.5
pithMetrics = pithEvaluation(realCoords, estCoords, testImages, alpha)

print("PROCENA CENTRA")
for key, val in pithMetrics.items():
    print("{}: {:.2f}".format(key, val))

# 2 Evaluacija broja prstenova
rings = list(zip(realNumOfRings, estNumOfRings))
firstFive, secondFive = TreeRingCountingEvaluation(rings, testImages)

print("\nPROCENA KRUGOVA NA DRVETU:")
print("\nOsnovna metrika")
for key, val in firstFive.items():
    print("{}: {:.2f}".format(key, val))

print("\nNaprednija metrika")
for key, val in secondFive.items():
    print("{}: {:.2f}".format(key, val))
```

PROCENA CENTRA  
MDE: 86.04  
STD: 96.47  
RMSE: 129.27  
Hit rate: 33.33

PROCENA KRUGOVA NA DRVETU:

Osnovna metrika  
ME: 8.33  
MAE: 11.67  
RMSE: 14.35  
MAPE: 55.70  
R2: -0.87

Naprednija metrika  
Precision: 0.72  
Recall: 0.94  
F1: 0.82  
Accuracy\_image: 0.00  
Accuracy\_ring: 0.94

## Analiza rezultata

### Procena centra (pith)

- MDE = 86.04: prosečna udaljenost detektovanog centra od stvarnog je prilično velika.
- STD = 96.47: greške su vrlo varijabilne među slikama.
- RMSE = 129.27: prisustvo ekstremnih odstupanja naglašava loše detekcije.
- Hit rate = 33.33%: samo trećina detekcija je unutar zadate tolerancije, preciznost je niska.

pith detekcija nije pouzdana i zahteva podešavanje parametara filtera, gradijenata i voting metode.

## Procena broja prstenova

### Osnovne metrike:

- ME = 8.33, MAE = 11.67, RMSE = 14.35: algoritam u proseku precenjuje broj prstenova, sa velikim greškama.
- MAPE = 55.70%: visoka relativna greška u odnosu na stvarni broj prstenova.
- $R^2 = -0.87$ : predikcije su lošije od prostog proseka stvarnih vrednosti.

### Napredne metrike:

- Precision = 0.72: 72% detektovanih prstenova je tačno, postoji nekoliko lažnih pozitivnih.
- Recall = 0.94: 94% stvarnih prstenova je uspešno detektovano.
- F1 = 0.82: harmonijska sredina Precision i Recall pokazuje solidnu, ali ne i savršenu detekciju.
- Accuracy\_image = 0.00: nijedna slika nije potpuno tačno detektovana.
- Accuracy\_ring = 0.94: na nivou pojedinačnih prstenova detekcija je vrlo dobra.

Algoritam precenjuje broj prstenova i ima varijabilnu preciznost, ali većina stvarnih prstenova je pravilno detektovana. Potrebno je fino podešavanje parametara detekcije vrhova i filtriranja za smanjenje prosečne greške.

---

## Uticaj parametara na performanse

### 1. Parametri pith detekcije (Hessian, beta, c, sigma)

- **Povećanje sigma (širina Gauss filtera):**
  - Glatka ridge mapa -> smanjuje šum, ali može zbiti detalje i pith biti pomeren.
  - MDE i RMSE obično rastu ako sigma postane prevelika.
- **Parametri beta i c (osetljivost filtera na oblik prstena):**
  - Prevelik beta -> naglašava samo tanke linije, šire prstenove može propustiti.
  - Prevelik c -> slabije razlikovanje pravih linija od šuma.
- **Uticaj na metrike:**
  - MDE i STD rastu kada pith nije precizno lociran.
  - Hit rate opada kada filter ne detektuje centar dovoljno tačno.

### 2. Parametri ROI ekstrakcije (roiPercent)

- **Mali ROI (%):**

- Brža obrada, manje nepotrebnog šuma, ali može propustiti deo prstenova -> ME i MAE rastu.
- **Veliki ROI (%)**:
  - Pokriva celu sliku -> precizniji broj prstenova, ali veća obrada, više šuma -> Precision može pasti.

### 3. Parametri detekcije prstenova (sigma, heightLevel, minProminence)

- **Sigma (glatkoća profila)**:
  - Prevelika sigma -> gubi se detalj, neki prstenovi se "spajaju" -> Recall opada, ME raste.
  - Premala sigma -> profil je šumovit, lažni vrhovi -> Precision opada, MAE raste.
- **heightLevel**:
  - Veći threshold -> samo najizraženiji prstenovi se detektuju -> Recall opada, Precision raste.
  - Manji threshold -> lakše se detektuju svi prstenovi, ali više lažnih -> Precision opada, F1 može pasti.
- **minProminence**:
  - Veći minProminence -> uklanja sitne nepravilnosti -> smanjuje lažne prstenove, Precision raste.
  - Premali -> više "šuma" prepoznato kao prsten -> ME i MAE rastu.

### 4. Procenjivanje validne vrednosti prstenova

- Uklanjanje odstupajućih detekcija smanjuje uticaj ekstremnih grešaka -> ME i MAE opadaju.